

T-SQL · SPARK SQL · DATABRICKS · FABRIC

The SQL Practitioner's Handbook 4.0

How SQL thinks and runs on Databricks and Fabric.

50 concepts, each with a figure like these: the mechanism behind it, drawn.

COMPANION

SQL Cheat Sheet 4.0

142 query patterns to keep open while you query.

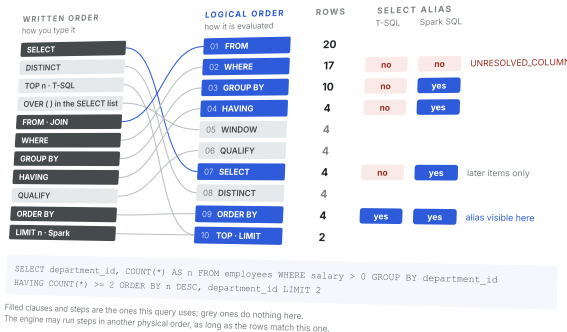
FREE

WEBSITE www.ssanjaychandra.com

GITHUB github.com/ssanjaychandra123

LINKEDIN linkedin.com/in/ssanjaychandra

Logical processing order



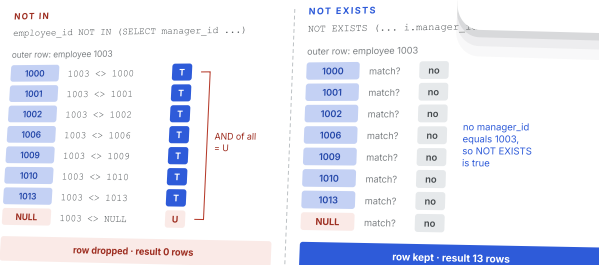
TE ...)

ry	RANGE (default)	ROWS
000	138,000	138,000
000	299,000	217,000 current row
000	299,000	299,000

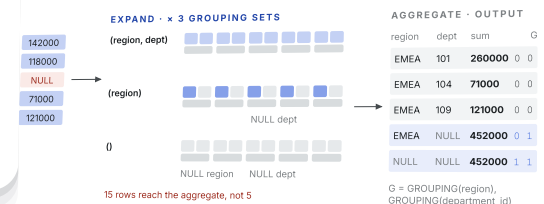
and every row tied with it

ROWS frame: up to this physical row; tie order is not defined

Anti joins and NOT IN



g sets



	BIN2_UTF8 default	CLASS
'London' = 'London'	false	true
'abc' = 'abc '	true	true
Sales vs sales as names	two names	one name
ORDER BY on A a B b	A B a b	case-blind

WHICH NULL IS WHICH - GROUP BY ROLLUP (MANAGER_ID)

	count 1	GROUPING = 0	a real group: the chief executive has no manager
NULL	count 20	GROUPING = 1	the grand total row added by ROLLUP

Warehouse pads trailing spaces before -, so the second row is true there; Spark compares bytes unless the collation has the RTRIM modifier. Names in the Warehouse follow its collation; Spark names are always case-insensitive.

Warehouse collation is fixed at creation. Spark collations: Spark 4.0, DBR 16.1.

The Periodic Table of SQL.

01 Lp Logical processing order	02 Nt NULL and three-valued logic	03 Bs Bags, sets and DISTINCT	04 Tp Type precedence and implicit conversion	05 Cc Collation and case sensitivity	06 Od Order and determinism	07 Jt Join types	08 Jc Join cardinality and fan-out	09 Sj Semi joins, anti joins and NOT IN	10 Wf Window functions	11 Wi Window frames: ROWS vs RANGE	12 Gs Grouping sets and ROLLUP	13 Sd Subqueries and decorrelation	14 Ci CTEs are inlined, not cached	15 Rq Recursive queries
16 Wa Warehouse architecture	17 Dp Distributed plans and data movement	18 Sc Statistics and cardinality estimates	19 Rs Result set caching	20 Wm Workload management and isolation	21 Rw Reading Warehouse plans and query insights	22 Hw How the Warehouse writes Delta	23 Si Snapshot isolation and write conflicts	24 Mt Multi-table transactions	25 Tt Time travel, clones and snapshots					
26 We Warehouse, endpoint and Lakehouse	27 Ws What the SQL analytics endpoint reads	28 Cd Cross-database queries	29 Sp SQL permissions vs OneLake security	30 DI Direct Lake fallback	31 Ip Ingestion paths	32 Sq SQL database and mirroring								
33 Sw SQL warehouse sizing and scaling	34 Rc Result caches	35 Qp The query profile	36 Ss SQL scripting and procedures	37 Ms Multi-statement transactions	38 Mv Metric views	39 Af AI functions in SQL								
40 Co Constraints the optimiser trusts	41 Is Identity and surrogate keys	42 St SCD Type 1	43 Sy SCD Type 2	44 Ot One table, two engines	45 Ps Parameters and SQL injection	46 Ts Testing SQL and data quality	47 Sl SQL in CI/CD	48 Wq What a query costs	49 Db Diagnose by symptom	50 Wh Which engine when				

■ 01 How SQL thinks
 ■ 02 The Fabric Warehouse engine
 ■ 03 Fabric SQL surfaces
 ■ 04 Databricks SQL
 ■ 05 Working across both engines
 ■ Back matter

THINKS

WAREHOUSE

FABRIC

DATABRICKS

ACROSS

INTRODUCTION

What this handbook is.

SQL on the two platforms most data teams build on today:

Microsoft Fabric and **Azure Databricks**. It is for engineers and architects who write SQL against both, from a first query in the Warehouse to deciding which engine should run it.

THIS HANDBOOK

Concepts, drawn

How SQL and the two engines behave. One concept per entry, each with a figure that shows the mechanism.

COMPANION

SQL Cheat Sheet 4.0

T-SQL and Spark SQL patterns side by side. Read the concept here once, keep the cheat sheet open while you query.

SCOPE AND VERSION

T-SQL as it runs in **Fabric Data Warehouse** and the **SQL analytics endpoint**, and Spark SQL on **Spark 4.1** as it runs on **Databricks Runtime 18** and **Fabric Runtime 2.0**, as of September 2026. Where a feature exists on one engine only, or behaves differently on an older runtime, the page says so.

HOW TO READ IT

Part 1 is about meaning and applies to both engines; read it first. Parts 2 to 4 cover one platform each, and Part 5 covers what happens when the same data meets both. Every entry stands alone: a lead under the title, a figure, then sections named after what you do with the concept.

COVERED IN THE PYSPARK PRACTITIONER'S HANDBOOK 4.0

These work the same way whether the query arrives as SQL or PySpark, so they have full pages there, not here.

Catalyst, AQE, pushdown and pruning	III Catalyst and query plans
Join strategies, Photon, NEE	IV Physical execution
Shuffle, skew, spill	V Shuffle, partitions, skew
Partitioning, Liquid Clustering, Z-Order, V-Order, OPTIMIZE, Predictive Optimization	VII Reading data and file layout
Delta log, ACID, concurrency, deletion vectors, time travel, VACUUM, change data feed, MERGE internals	VIII Delta Lake mechanics
Materialised views, streaming tables, Lakeflow pipelines, AUTO CDC, Unity Catalog, row filters and column masks	X Running PySpark on Databricks
OneLake, shortcuts, materialised lake views, capacity and throttling	XI Running PySpark on Fabric
Classic vs serverless compute	XII Compute choices, utilities and testing

CONVENTIONS

Part 1 · Blue How SQL thinks	Part 2 · Green The Fabric Warehouse engine	Part 3 · Fuchsia Fabric SQL surfaces
Part 4 · Teal Databricks SQL	Part 5 · Blue grey Working across both engines	Red Cost, failure, path to avoid

In Part 1, the **clause strip** under each title marks the steps of a query the concept acts in, in logical order from *FROM* to *LIMIT*. From Part 2 on, a **layer strip** shows what the concept acts on: Storage, Table, Model, Load, Query, Compute, Govern, Operate. In the **figures**, the part colour marks the concept and its good path, grey is context, and red flags cost, failure, or the path to avoid.

CONTENTS

Where each chapter acts.

Concept index 05 Conclusion 56

	CLAUSE · PART 1	FROM	WHERE	GROUP BY	HAVING	WINDOW	SELECT	ORDER BY	LIMIT	CONCEPTS	PAGE
01 HOW SQL THINKS											
I	Query semantics	5	4	4	2	2	4	4	2	6	07
II	Relational operations	6	2	1		2	3	1		9	12
	LAYER · PARTS 2 TO 5	STORAGE	TABLE	MODEL	LOAD	QUERY	COMPUTE	GOVERN	OPERATE		
02 THE FABRIC WAREHOUSE ENGINE											
III	How the Warehouse runs a query	1	1			5	4		2	6	21
IV	Warehouse tables and transactions	2	4		3				1	4	26
03 FABRIC SQL SURFACES											
V	Fabric SQL surfaces	2	2	1	2	5		2	1	7	31
04 DATABRICKS SQL											
VI	Databricks SQL		1	1	2	4	3	1	3	7	38
05 WORKING ACROSS BOTH ENGINES											
VII	Modelling across both engines		4	4	3	2				4	45
VIII	Operating across both engines	1	1		1	2	1	1	3	5	48
BACK MATTER											
IX	Reference									2	54

Cell number = concepts in the chapter at that clause (Part 1) or layer (Parts 2 to 5); darker = a larger share of the chapter.

Concept index: clause and layer view.

PART 1 · CLAUSE

1 FROM 2 WHERE 3 GROUP BY 4 HAVING 5 WINDOW 6 SELECT 7 ORDER BY 8 LIMIT

PARTS 2 TO 5 · LAYER

A Storage B Table C Model D Load E Query F Compute G Govern H Operate

CONCEPT

I · QUERY SEMANTICS

	1	2	3	4	5	6	7	8
Logical processing order								
NULL and three-valued logic								
Bags, sets and DISTINCT								
Type precedence and implicit conversion								
Collation and case sensitivity								
Order and determinism								

II · RELATIONAL OPERATIONS

	1	2	3	4	5	6	7	8
Join types								
Join cardinality and fan-out								
Semi joins, anti joins and NOT IN								
Window functions								
Window frames: ROWS vs RANGE								
Grouping sets and ROLLUP								
Subqueries and decorrelation								
CTEs are inlined, not cached								
Recursive queries								

III · HOW THE WAREHOUSE RUNS A QUERY

	A	B	C	D	E	F	G	H
Warehouse architecture								
Distributed plans and data movement								
Statistics and cardinality estimates								
Result set caching								
Workload management and isolation								
Reading Warehouse plans and query insights								

IV · WAREHOUSE TABLES AND TRANSACTIONS

	A	B	C	D	E	F	G	H
How the Warehouse writes Delta								
Snapshot isolation and write conflicts								
Multi-table transactions								
Time travel, clones and snapshots								

CONCEPT

V · FABRIC SQL SURFACES

	A	B	C	D	E	F	G	H
Warehouse, endpoint and Lakehouse								
What the SQL analytics endpoint reads								
Cross-database queries								
SQL permissions vs OneLake security								
Direct Lake fallback								
Ingestion paths								
SQL database and mirroring								

VI · DATABRICKS SQL

	A	B	C	D	E	F	G	H
SQL warehouse sizing and scaling								
Result caches								
The query profile								
SQL scripting and procedures								
Multi-statement transactions								
Metric views								
AI functions in SQL								

VII · MODELLING ACROSS BOTH ENGINES

	A	B	C	D	E	F	G	H
Constraints the optimiser trusts								
Identity and surrogate keys								
SCD Type 1								
SCD Type 2								

VIII · OPERATING ACROSS BOTH ENGINES

	A	B	C	D	E	F	G	H
One table, two engines								
Parameters and SQL injection								
Testing SQL and data quality								
SQL in CI/CD								
What a query costs								

IX · REFERENCE

	A	B	C	D	E	F	G	H
Diagnose by symptom								
Which engine when								

01

How SQL thinks.

Before any engine is involved, a query has a meaning: which rows it keeps, how many each join returns, where NULL goes and what order comes back. Fabric Warehouse and Spark SQL agree on most of it and differ in places that change results: integer division, trailing spaces, where an alias is visible, whether a recursive query runs at all. These two chapters cover that meaning, with the differences marked on every page.

I	Query semantics	
01	Logical processing order	07
02	NULL and three-valued logic	08
03	Bags, sets and DISTINCT	08
04	Type precedence and implicit conversion	09
05	Collation and case sensitivity	10
06	Order and determinism	11
II	Relational operations	
07	Join types	12
08	Join cardinality and fan-out	13
09	Semi joins, anti joins and NOT IN	13
10	Window functions	14
11	Window frames: ROWS vs RANGE	16
12	Grouping sets and ROLLUP	17
13	Subqueries and decorrelation	17
14	CTEs are inlined, not cached	18
15	Recursive queries	19

READING PATH

Clause heat: 1 2 3 4 5 6 7 8 = FROM to LIMIT

I

What does this query mean, and which rows does it keep?



II

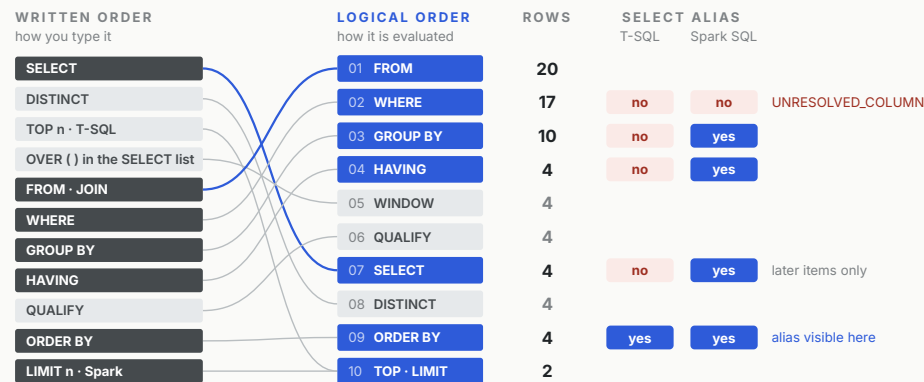
How many rows does each operation return, and why?



Logical processing order.

CLAUSE FROM WHERE GROUP BY HAVING WINDOW SELECT ORDER BY LIMIT

You write a query in one order: **SELECT** with **DISTINCT** and T-SQL's **TOP**, then **FROM**, **WHERE**, **GROUP BY**, **HAVING**, **QUALIFY**, **ORDER BY** and Spark's **LIMIT**. It is evaluated in another, fixed logical order: **FROM** and the joins, **WHERE**, **GROUP BY**, **HAVING**, window functions, **QUALIFY**, **SELECT**, **DISTINCT**, **ORDER BY**, then **TOP** or **LIMIT**. Each step sees only what the steps before it produced. That order decides which names are in scope and what the query means; the engine is free to execute it in any physical order that returns the same rows.



```
SELECT department_id, COUNT(*) AS n FROM employees WHERE salary > 0 GROUP BY department_id
HAVING COUNT(*) >= 2 ORDER BY n DESC, department_id LIMIT 2
```

Filled clauses and steps are the ones this query uses; grey ones do nothing here.
The engine may run steps in another physical order, as long as the rows match this one.

FIG. 1.1 Left, the order you write a SELECT in; right, the order it is evaluated in. Each line joins a clause to its logical step. Beside each step, the rows left after it for one query on the 20-row employees table, and where a SELECT alias can be referenced: T-SQL only in ORDER BY, Spark SQL also in GROUP BY, HAVING and later SELECT items.

WHY AN ALIAS FAILS IN WHERE

An alias is born in **SELECT**, so earlier steps cannot see it. T-SQL is strict: a **SELECT** alias works in **ORDER BY** on its own, and nowhere before it, so **WHERE s > 100000** fails with invalid column name. Spark SQL is looser. It resolves a **SELECT** alias in **GROUP BY** (`spark.sql.groupByAliases`, on by default) and in **HAVING**, and since Spark 3.4 a later **SELECT** item can use an earlier alias, such as `salary * 1.1 AS s, s - salary AS raise`. **WHERE** still fails in both; Spark reports `UNRESOLVED_COLUMN.WITH_SUGGESTION`. Code written against Spark's rules breaks when you move it to the Warehouse.

FILTERING ON A WINDOW

Window functions run after **WHERE** and **HAVING**, so neither can filter on `ROW_NUMBER()`. Wrap the query in a subquery and filter outside it, or use **QUALIFY**, the step that sits after the window functions. Fabric Warehouse and the SQL analytics endpoint support **QUALIFY**, and so does Databricks; open-source Spark adds it in 4.2, and Spark 4.1 rejects it.

ON OR WHERE IN AN OUTER JOIN

For an inner join it makes no difference whether a condition sits in **ON** or **WHERE**. For a **LEFT JOIN** it decides the result. **ON** runs as part of the join, so a filter on the right table there keeps every left row and blanks the right side when nothing matches. The same filter in **WHERE** runs after the join, meets **NULL** in those blanked rows and removes them, which quietly turns the query into an inner join.

WHAT ACTUALLY RUNS

Logical order is not execution order. Both optimisers push **WHERE** predicates below joins and into the scan, where they can skip whole files; in Spark you see them as **PushedFilters** in **EXPLAIN**. The one constraint is that every rewrite must return what the logical order would, which is why neither engine moves a **WHERE** filter into the **ON** of an outer join. **GROUP BY ALL** reads the **SELECT** list to find its keys and works in Fabric Warehouse, Databricks and Spark 4.1; **ORDER BY ALL** sorts by every **SELECT** item in the same three.

RELATED CONCEPTS

NULL and three-valued logic, Window frames: ROWS vs RANGE, Subqueries and decorrelation, Order and determinism.

NULL and three-valued logic.

CLAUSE FROM WHERE GROUP BY HAVING WINDOW SELECT ORDER BY LIMIT

A comparison with `NULL` is neither true nor false; it is unknown. `WHERE`, `ON`, `HAVING` and `CASE WHEN` keep a row only when the condition is true, so unknown rows disappear without an error. Fabric Warehouse and Spark SQL follow the same rules here; they differ only in syntax.

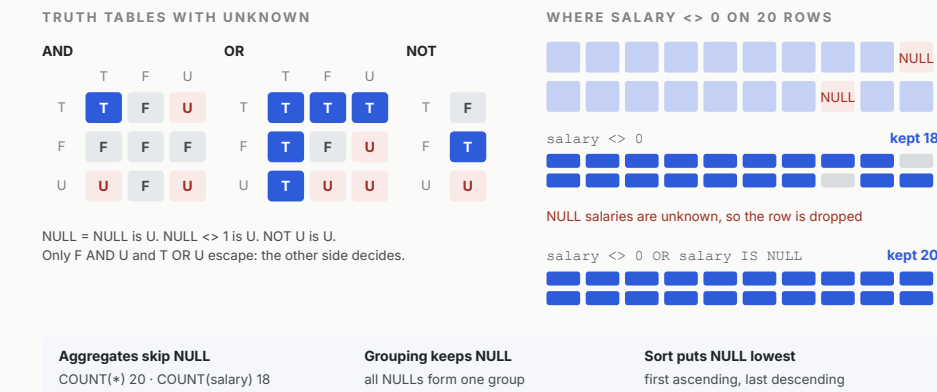


FIG. 1.2 Three-valued logic: any comparison with NULL is unknown, and WHERE keeps only rows that are true. On the 20 salaries, salary <> 0 keeps 18 because the two NULL salaries are unknown, not unequal.

WHERE ROWS GO MISSING

In a 20-row employees table with two missing salaries, `WHERE salary <> 0` returns 18 rows: the two `NULL` salaries are unknown, not unequal. `NOT (salary = 0)` is still unknown, and `salary = NULL` never matches anything; write `IS NULL`. `AND` and `OR` follow a truth table of their own: `NULL AND FALSE` is false and `NULL OR TRUE` is true, because the other side already decides; every other mix with unknown stays unknown. When you mean "different, and a missing value counts as different", say so with `salary <> 0 OR salary IS NULL`.

COUNTING AND GROUPING

`COUNT(*)` counts rows and `COUNT(salary)` counts non-null values: 20 against 18. `SUM` and `AVG` skip `NULL`, so the average salary is 114,500 over 18 people, or 103,050 if you count the missing two as zero with `COALESCE`. Decide which one the report means. `SUM` over only `NULL` values returns `NULL`, not 0. `GROUP BY`, `DISTINCT` and the set operators go the other way and treat every `NULL` as the same value, so all missing salaries land in one group.

COMPARING SAFELY

`IS [NOT] DISTINCT FROM` always returns true or false and treats two `NULL`s as equal; Fabric Warehouse and Spark both support it, and Spark also spells it `<=>`. It matters in joins: a self-join on `manager_id` returns 77 rows with `=` and 78 with `<=>`, the extra row being the chief executive, whose manager is `NULL`, matching herself. Use it deliberately, because a join that matches `NULL` keys can multiply rows.

SORTING NULL

Both engines treat `NULL` as the lowest value: first in ascending order, last in descending. Spark lets you override that with `NULLS FIRST` or `NULLS LAST`. Fabric Warehouse has no such syntax, so sort on `CASE WHEN salary IS NULL THEN 1 ELSE 0 END` first when you want missing values at the bottom of an ascending list.

RELATED CONCEPTS

Semi joins, anti joins and NOT IN, Bags, sets and DISTINCT, Logical processing order, Grouping sets and ROLLUP.

Bags, sets and DISTINCT.

CLAUSE
FROM
WHERE
GROUP BY
HAVING
WINDOW
SELECT
ORDER BY
LIMIT

A SQL table is a bag: the same row can appear more than once, and neither Fabric Warehouse nor Delta enforces uniqueness for you. Joins, `UNION ALL` and `GROUP BY` keep that multiplicity. `DISTINCT`, `UNION`, `INTERSECT` and `EXCEPT` turn the result into a set, which changes both the row count and the cost.

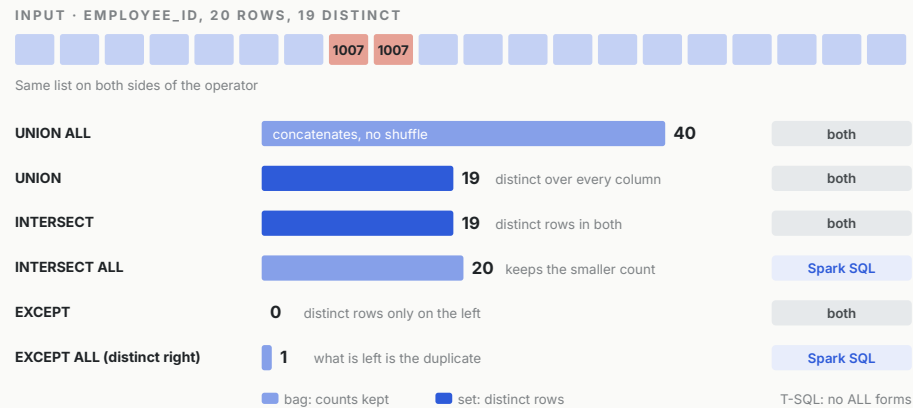


FIG. 1.3 Two copies of the 20 employee ids, where id 1007 appears twice, combined with each set operator. Bag operators keep counts; set operators return distinct rows. INTERSECT ALL and EXCEPT ALL exist only in Spark SQL.

WHERE DUPLICATES HIDE

A Fabric Warehouse primary key is declared `NOT ENFORCED`, and a Databricks primary key is informational, so neither stops a second copy of a row from being written. `DISTINCT` removes only rows that match in every column. If employee 1007 was loaded twice with two different salaries, `SELECT DISTINCT *` still returns both rows: 20 rows for 19 people. Removing that kind of duplicate needs a rule, such as keeping the row with the highest salary with `ROW_NUMBER()` per key.

SET OPERATORS

Stack the 20 employee ids on themselves: `UNION ALL` returns 40 rows, `UNION` returns 19. `INTERSECT` and `EXCEPT` also return distinct rows in both engines, and they compare `NULL` with `NULL` as equal. Spark adds the bag versions, `INTERSECT ALL` and `EXCEPT ALL`, which keep counts. They make a quick duplicate finder: the 20 ids `EXCEPT ALL` the 19 distinct ids leaves one row, 1007. T-SQL has no `ALL` form of either; group by the key with `HAVING COUNT(*) > 1` instead.

WHAT DEDUPLICATION COSTS

`UNION ALL` just concatenates its inputs. `UNION` and `DISTINCT` add an aggregation over every output column, which in Spark means a `HashAggregate` and a shuffle of all the rows on all the columns. When the inputs cannot overlap, such as two different months, write `UNION ALL`. For counting, an exact `COUNT(DISTINCT x)` has to see every distinct value; `APPROX_COUNT_DISTINCT` in the Warehouse guarantees up to a 2% error rate within a 97% probability, and `approx_count_distinct` in Spark does the same job for dashboards and large tables.

RELATED CONCEPTS

Join cardinality and fan-out, NULL and three-valued logic, Grouping sets and ROLLUP, Order and determinism.

Type precedence and implicit conversion.

CLAUSE FROM WHERE GROUP BY HAVING WINDOW SELECT ORDER BY LIMIT

When an expression mixes types, both engines convert one side before they compute, without telling you. T-SQL converts the lower type to the higher one on a fixed precedence list. Spark applies its own coercion rules, which are stricter under ANSI mode, the default since Spark 4.0. The same expression can return a different value, a different type, or an error on one engine and a result on the other.

SAME QUERY, DIFFERENT ANSWER

`5 / 2` is 2 in T-SQL, because an integer divided by an integer truncates, and 2.5 in Spark, where `/` always returns a fractional type; Spark's integer division is `5 div 2`. `AVG` over an `INT` column returns `int` in T-SQL, so three salaries of 96,000, 92,000 and 150,000 average 112,666 there and 112,666.67 in Spark. `SUM` of an `INT` column returns `int` in T-SQL and fails once the total passes 2,147,483,647; Spark returns `BIGINT`. `'1' + '1'` is the string `'11'` in T-SQL and an error in Spark, `DATATYPE_MISMATCH.BINARY_OP_WRONG_TYPE`, while `'1' + 1` is 2 in both.

HOW T-SQL DECIDES

The type with the lower precedence converts to the higher one: `datetime2` above `decimal`, above `bigint`, above `int`, above `varchar`. Comparing a `varchar` column with an integer literal therefore converts every value in the column, and one row holding `n/a` fails the whole query with a conversion error. Decimal results cap at a precision of 38. When a multiplication or division would need more, T-SQL shrinks the scale to protect the integer part, down to `decimal(38, 6)`.

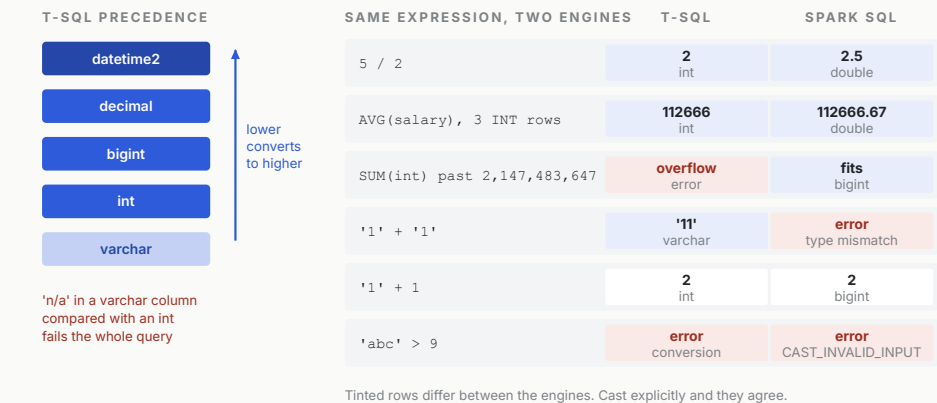


FIG. 1.4 The same expressions evaluated by the two engines. T-SQL converts the lower type to the higher one on its precedence list and keeps integer arithmetic in integers; Spark under ANSI mode returns fractional results from division and averages, and rejects + on two strings.

HOW SPARK DECIDES

Under ANSI mode Spark casts a string used in arithmetic or compared with a number to `BIGINT`, so `'10' > 9` is true and `'abc' > 9` fails at run time with `CAST_INVALID_INPUT` on the row that holds it. Decimal arithmetic also caps at 38 digits: `DECIMAL(38,10) * DECIMAL(38,10)` comes back as `decimal(38,6)`, because Spark keeps at least six digits of scale when it has to drop some. A `UNION ALL` of `1` and `1.5` widens both rows to `decimal(11,1)`.

WRITING IT ONCE

Cast explicitly and the two engines agree. Cast integers to `DECIMAL` before you average or divide them, write `FLOOR(a / b)` when you want integer division on both, and compare dates with typed values instead of strings. Check the result type with `typeof()` in Spark; in the Warehouse, create a table from the expression with CTAS and read its type in `INFORMATION_SCHEMA.COLUMNS`.

RELATED CONCEPTS

Collation and case sensitivity, NULL and three-valued logic, Bags, sets and DISTINCT.

Collation and case sensitivity.

CLAUSe

FROM

WHERE

GROUP BY

HAVING

WINDOW

SELECT

ORDER BY

LIMIT

Collation decides whether `'London'` equals `'london'`, how text sorts, and whether a trailing space counts. Fabric Warehouse and Spark both compare text case-sensitively by default. They still disagree on trailing spaces and on whether table and column names are case-sensitive, and that is where results drift between the two.

	Warehouse BIN2_UTF8 default	Warehouse CLAS=UTF8	Spark UTF8_BINARY default	Spark UTF8_LCASE	Spark RTRIM modifier
<code>'London' = 'london'</code>	false	true	false	true	false
<code>'abc' = 'abc '</code>	true	true	false	false	true
Sales vs sales as names	two names	one name	one name	one name	one name
ORDER BY on A a B b	A B a b	case-blind	A B a b	a A b B	A B a b

Warehouse pads trailing spaces before =, so the second row is true there; Spark compares bytes unless the collation has the RTRIM modifier.
Names in the Warehouse follow its collation; Spark names are always case-insensitive.

Warehouse collation is fixed at creation. Spark collations: Spark 4.0, DBR 16.1.

FIG. 1.5 Four comparisons under the two Fabric Warehouse collations and three Spark collations. Defaults are marked. The engines agree on case but not on trailing spaces or on whether table and column names are case-sensitive.

IN FABRIC WAREHOUSE

New warehouses and every SQL analytics endpoint take the workspace's default collation, which starts as `Latin1_General_100_BIN2_UTF8`: binary and case-sensitive. The only other supported collation is `Latin1_General_100_CI_AS_KS_WS_SC_UTF8`, case-insensitive. Set it before you create the warehouse, in the workspace setting or through the REST API, because it cannot be changed afterwards. Names follow the collation too: under the default, `sales` and `Sales` are two different schemas. In a `SELECT` you can apply any `COLLATE` to one expression; `CREATE TABLE` and CTAS accept only the two supported ones.

IN SPARK AND DATABRICKS

String comparison defaults to `UTF8_BINARY`. Collations arrived in Spark 4.0 and Databricks Runtime 16.1: `UTF8_LCASE`, `UNICODE`, `UNICODE_CI` and locale-specific ones, applied per expression with `COLLATE` or as a default for a schema or table on Databricks. Names are the opposite of the Warehouse default: Spark resolves `EMPLOYEE_ID FROM Employees` without complaint.

TRAILING SPACES

T-SQL pads the shorter string before comparing, so `'abc' = 'abc '` is true; only `LIKE` treats a trailing space in the pattern as significant. Spark compares the bytes, so the same test is false, unless you use the `RTRIM` modifier, as in `UTF8_BINARY_RTRIM`, from Databricks Runtime 16.2. A name loaded as `'Carlos Mendes '` therefore joins in the Warehouse and misses in Spark. `LEN` ignores trailing spaces; Spark's `length` counts them.

SORTING AND GROUPING

A binary collation sorts by code point, so upper case comes first: `A, B, a, b`. `UTF8_LCASE` sorts `a, A, b, B`, and grouping under it returns one row per value ignoring case. Normalise join and grouping keys with `UPPER` or `TRIM` when data from both engines meets, instead of relying on either default.

RELATED CONCEPTS

Type precedence and implicit conversion, Join cardinality and fan-out, Bags, sets and DISTINCT.

Order and determinism.

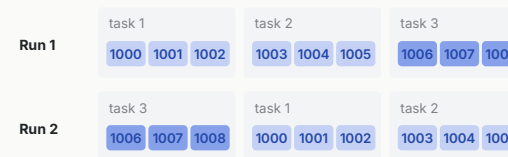
CLAUSE FROM WHERE GROUP BY HAVING WINDOW SELECT ORDER BY LIMIT

A result has no order unless the outermost query has an `ORDER BY`. Rows arrive in whatever order the tasks finish, and that can change between runs, retries and cluster sizes. The same applies to `TOP` or `LIMIT` without a unique sort key, and to functions that return a new value each time.

WHERE ORDER GETS LOST

An `ORDER BY` inside a view, CTE, derived table or subquery promises nothing about the final result. T-SQL rejects it unless `TOP` or `OFFSET` comes with it. Spark accepts it and may simply drop it: sort a subquery, aggregate over it, and the sort is gone from the plan. Put the sort where the rows leave the query.

NO ORDER BY · ROWS ARRIVE AS TASKS FINISH

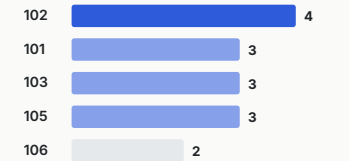


Same rows, different order. Only the outermost ORDER BY fixes it.

Make it repeatable

End every ORDER BY with a unique column:
ORDER BY n DESC, department_id
Do the same inside OVER (ORDER BY ...) when a ROW_NUMBER picks the row a dedup keeps.

ORDER BY HEADCOUNT DESC LIMIT 2



tie on 3: the second row is any of them

Run 1	102 · 103
Run 2	102 · 101
TOP (2) WITH TIES	102 · 101 · 103 · 105

Spark has no WITH TIES: filter on RANK()

FIG. 1.6 The same query run twice. Tasks finish in a different order, so rows without ORDER BY come back in a different order, and LIMIT 2 over a tie on 3 returns a different second department. TOP WITH TIES returns every tied row instead.

TIES

Ask for the two largest departments by headcount and the first row is 102 with four people. The second is any of 101, 103 or 105, which tie on three, and which one you get depends on the run. `TOP (2) WITH TIES` in the Warehouse returns all tied rows instead, in no fixed order among themselves. Spark has no `WITH TIES`; rank with `RANK()` and filter on the rank. `ROW_NUMBER()` has the same problem: with a non-unique `ORDER BY` inside `OVER`, a dedup query can keep a different row each time it runs, so a reload does not produce the same table.

MAKING IT REPEATABLE

Add a unique column as the last sort key, in the final `ORDER BY` and in every `OVER (ORDER BY ...)` that picks a row: `ORDER BY salary DESC, employee_id`. Paging with `LIMIT` and `OFFSET` needs that total order too, or rows repeat and vanish between pages. `any_value` and `first` without an order return whichever row arrives first.

FUNCTIONS THAT CHANGE

In Spark, `rand()` and `uuid()` produce a new value for every row, while `current_timestamp()` is fixed once per query: 100,000 rows get one timestamp. In T-SQL, `GETDATE` and `SYSDATETIME` are documented as nondeterministic. Anything that depends on them, or on the order rows arrive, will not reproduce exactly, which matters for tests and for loads you expect to be idempotent.

RELATED CONCEPTS

Logical processing order, Window frames: ROWS vs RANGE, CTEs are inlined, not cached, Bags, sets and DISTINCT.

Join types.

CLAUSE **FROM** WHERE GROUP BY HAVING WINDOW SELECT ORDER BY LIMIT

A join pairs rows from two tables on a condition. The join type decides what happens to a row that finds no partner: an inner join drops it, an outer join keeps it and fills the other side with `NULL`, and a cross join skips the condition and pairs everything.

Joining 20 employees to 10 departments returns 19, 20, 20, 21 or 200 rows depending on the type.

SAMPLE · E = EMPLOYEES, D = DEPARTMENTS

Aisha Bello	102	102	Data & Analytics
Samuel Otieno	105	105	Finance
Nina Petrova	111		Cybersecurity

INNER JOIN		2 rows · 19 on the full tables			
e	d	employee	e.dept	d.dept	department
102	102	Aisha	102	102	Data & Analytics
105	105	Samuel	105	105	Finance
111	108				

matches only

LEFT JOIN		3 rows · 20 on the full tables			
e	d	employee	e.dept	d.dept	department
102	102	Aisha	102	102	Data & Analytics
105	105	Samuel	105	105	Finance
111	108	Nina	111		NULL

every row of e

RIGHT JOIN		3 rows · 20 on the full tables			
e	d	employee	e.dept	d.dept	department
102	102	Aisha	102	102	Data & Analytics
105	105	Samuel	105	105	Finance
111	108			108	Cybersecurity

every row of d

FULL JOIN		4 rows · 21 on the full tables			
e	d	employee	e.dept	d.dept	department
102	102	Aisha	102	102	Data & Analytics
105	105	Samuel	105	105	Finance
111	108	Nina	111		NULL
				108	Cybersecurity

every row of both

■ key kept ■ key dropped ■ looked up, columns not returned ■ NULL added by the join

FIG. 2.1A Three employees and three departments from the full tables, joined on `department_id`. Each panel shows which keys survive, the lines that match, and the rows that come out. Outer joins keep a row with no partner and fill the other side with `NULL`.

CROSS JOIN		9 rows · 200 on the full tables			
e	d	employee	e.dept	d.dept	department
102	102	Aisha	102	102	Data & Analytics
105	105	Samuel	105	105	Finance
111	108	Nina	111	108	Cybersecurity

no ON condition
each cell is one output row: $3 \times 3 = 9$

LEFT SEMI JOIN		2 rows · 19 on the full tables			
e	d	employee	e.dept	d.dept	department
102	102	Aisha	102		
105	105	Samuel	105		
111	108				

T-SQL: WHERE EXISTS

LEFT ANTI JOIN		1 row · 1 on the full tables			
e	d	employee	e.dept	d.dept	department
102	102				
105	105				
111	108	Nina	111		

T-SQL: WHERE NOT EXISTS

SELF JOIN (LEFT)		3 rows · 20 on the full tables			
e.mgr	m.id	employee	manager_id	manager	
NULL	1000	Grace	NULL	NULL	
1000	1006	Aisha	1000	Grace	
1006	1003	Priya	1006	Aisha	

m = employees again

FIG. 2.1B The same sample. A cross join pairs every employee with every department. Semi and anti joins use the departments only to decide which employees to keep and return the employee columns alone. A self join matches employees to their managers in the same table.

WHICH ROWS SURVIVE

Nineteen employees work in a department that exists, so the inner join returns 19 rows; the contractor in department 111 has no match. **LEFT JOIN** keeps her with `NULL` department columns: 20. **RIGHT JOIN** instead keeps Cybersecurity, department 108, which has nobody: 20. **FULL JOIN** keeps both: 21. **CROSS JOIN** pairs every employee with every department: 200. Semi and anti joins return only the left table's columns: the 19 employees with a department, or the one without.

NULLS THE JOIN ADDS

The `NULL`s an outer join adds look exactly like missing values in the data. To count matches, count a column from the optional side, `COUNT(d.department_id)`, not `COUNT(*)`. In a **FULL JOIN** the key can come from either side, so write `COALESCE(e.department_id, d.department_id)`; Spark's `USING (department_id)` does that for you and returns 111 and 108 in one key column.

CHAINING JOINS

Each join works on the result so far. Join employees to their managers with **LEFT JOIN**, then add an inner join to departments on the manager's `department_id`, and the chief executive disappears again: her manager columns are `NULL`, so the inner join has nothing to match, and 20 rows become 19. Keep every later join to the optional side an outer join, and put conditions on that side in **ON**, not **WHERE**.

SYNTAX THAT DIFFERS

Spark accepts `USING` and `NATURAL JOIN`; T-SQL has neither, so write the condition with `ON`. For a join that runs a subquery per row, such as the top earner of each department, T-SQL uses `CROSS APPLY` and `OUTER APPLY`, and Spark uses `LATERAL`. Spark names the join type in its plan, as in `BroadcastHashJoin LeftOuter`; which algorithm it picks is a separate decision made by the planner.

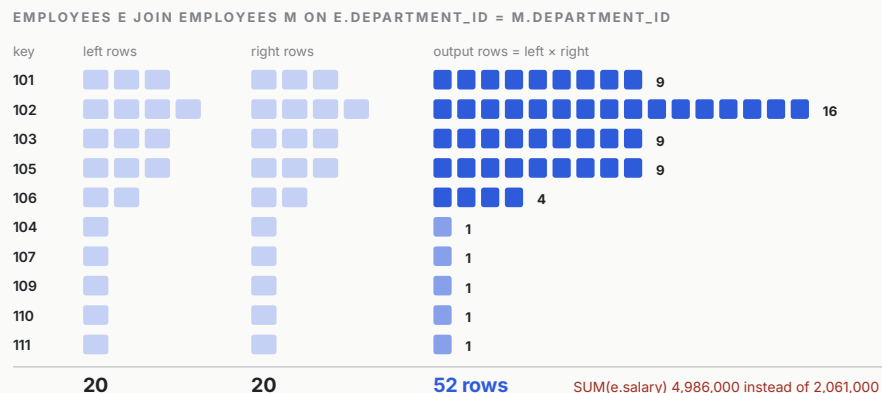
RELATED CONCEPTS

Join cardinality and fan-out, Semi joins, anti joins and NOT IN, Logical processing order, NULL and three-valued logic.

Join cardinality and fan-out.

CLAUSE **FROM** WHERE GROUP BY HAVING WINDOW SELECT ORDER BY LIMIT

A join returns one row for every pair of rows that match, not one row per row on the left. When a key repeats on both sides, the row count multiplies, and any total you then compute is inflated. Neither engine warns you: the query is valid and only the numbers are wrong.



One duplicate is enough: employee 1007 appears twice, so a join on `manager_id` lists Mark Lin twice under his manager.

FIG. 2.2 Employees joined to employees on `department_id`. Each key produces left count times right count rows, so 20 rows become 52 and a SUM over the result reports 4,986,000 for a payroll of 2,061,000.

READ THE RELATIONSHIP

Before you join, say which side is unique on the key. Many-to-one keeps the row count of the many side: employees joined to departments gives one row per employee who has a department. Many-to-many multiplies: employees joined to employees on `department_id` turns 20 rows into 52, and a `SUM(salary)` over the result reports 4,986,000 for a payroll of 2,061,000. For each key the output has left count times right count rows.

THE HIDDEN DUPLICATE

One repeated key is enough. Employee 1007 appears twice, so a join on `manager_id` lists Mark Lin twice under his manager, who then seems to manage two people instead of one, and a join on `employee_id` doubles him in every report. Check before you trust a total: compare `COUNT(*)` before and after the join, and find repeated keys with `GROUP BY key HAVING COUNT(*) > 1`.

AGGREGATE, THEN JOIN

When two facts meet one dimension, such as orders and payments for the same customer, joining both at once pairs every order with every payment. Sum each fact per customer first, then join the two summaries. The same applies to a lookup that is not unique: deduplicate it before it becomes the right side of a join.

WHAT THE ENGINES ASSUME

Declared keys do not protect you. A Fabric Warehouse primary key is `NOT ENFORCED` and a Databricks one is informational, so a table can hold duplicates while its metadata says otherwise. Anything that relies on the declared key, a semantic model relationship or a join you assumed was one-to-one, inherits the duplicate.

RELATED CONCEPTS

Bags, sets and DISTINCT, Semi joins, anti joins and NOT IN, NULL and three-valued logic.

Semi joins, anti joins and NOT IN.

CLAUSE **FROM** **WHERE** GROUP BY HAVING WINDOW SELECT ORDER BY LIMIT

`EXISTS` and `IN` keep a row when a match exists; `NOT EXISTS` and `NOT IN` keep it when none does. They look interchangeable. `NOT IN` follows three-valued logic, though: one `NULL` in the list makes every comparison unknown, and the query returns nothing.

NOT IN

employee_id NOT IN (SELECT manager_id ...)

outer row: employee 1003

1000	1003 <> 1000	T
1001	1003 <> 1001	T
1002	1003 <> 1002	T
1006	1003 <> 1006	T
1009	1003 <> 1009	T
1010	1003 <> 1010	T
1013	1003 <> 1013	T
NULL	1003 <> NULL	U

AND of all = U

row dropped · result 0 rows

NOT EXISTS

NOT EXISTS (... i.manager_id = o.employee_id)

outer row: employee 1003

1000	match?	no
1001	match?	no
1002	match?	no
1006	match?	no
1009	match?	no
1010	match?	no
1013	match?	no
NULL	match?	no

no manager_id equals 1003, so NOT EXISTS is true

row kept · result 13 rows

FIG. 2.3 Finding employees who manage nobody. NOT IN compares each id with every manager_id; the NULL manager of the chief executive makes every comparison unknown, so no row survives. NOT EXISTS only asks whether a matching row exists and returns 13.

THE NULL TRAP

Find employees who manage nobody. `WHERE employee_id NOT IN (SELECT manager_id FROM employees)` returns 0 rows, because the chief executive has no manager and her NULL is in the list. `NOT EXISTS` with `WHERE i.manager_id = o.employee_id` returns the right answer, 13, and so does `NOT IN` once the subquery adds `WHERE manager_id IS NOT NULL`. Both engines behave the same way, because the standard says so. Use `NOT EXISTS` by default.

EACH ROW ONCE

A semi join returns each left row at most once, however many matches it has. Departments that have staff: 9 rows with `EXISTS`, while an inner join to employees returns 19. An anti join is the mirror image. Spark accepts `LEFT SEMI JOIN` and `LEFT ANTI JOIN` as syntax; T-SQL has no such keywords, so write `EXISTS` and `NOT EXISTS`.

WHAT THE PLAN SHOWS

Spark rewrites `EXISTS` and `IN` as `LeftSemi` joins and `NOT EXISTS` as `LeftAnti`. A single-column `NOT IN` becomes a null-aware anti join: a `BroadcastHashJoin LeftAnti` that also tracks whether the subquery produced a NULL. A `NOT IN` over several columns cannot use it and falls back to a `BroadcastNestedLoopJoin` with an `OR isnull(...)` condition per column, which compares every row with every row. On large inputs that is the slow plan to look out for.

RELATED CONCEPTS

NULL and three-valued logic, Join cardinality and fan-out, Subqueries and decorrelation.

Window functions.

CLAUSE FROM WHERE GROUP BY HAVING WINDOW SELECT ORDER BY LIMIT

A window function computes a value for each row from a set of related rows, and keeps every row. `GROUP BY` collapses a department into one row; `SUM(salary)` `OVER (PARTITION BY department_id)` puts the department total on every employee. `OVER` has three parts: `PARTITION BY` splits the rows into groups, `ORDER BY` sorts inside each group, and the frame picks which of those rows the function sees.

GROUP BY DEPARTMENT_ID			OVER (PARTITION BY DEPARTMENT_ID ORDER BY SALARY DESC)				
102	4 rows	338,000	employee	salary	ROW_NUMBER	gap = salary - LAG	dept total
105	3 rows	299,000	Aisha Bello	150,000	1	no previous row	338,000
			Priya Nair	96,000	2	-54,000	338,000
			Robert Ramirez	92,000	3	-4,000	338,000
			Omar Haddad	NULL	4	NULL	338,000
2 rows out: the detail is gone			102				
			Samuel Otieno	138,000	1	no previous row	299,000
			Mark Lin	82,000	2	-56,000	299,000
			Mark Lin	79,000	3	-3,000	299,000
			105				
7 rows out: every row kept, values computed within its partition							

THE THREE PARTS OF OVER

PARTITION BY splits rows into groups **ORDER BY** sorts each group **frame** rows the function sees

Ranking functions, LAG and LEAD ignore the frame; SUM, AVG, FIRST_VALUE and LAST_VALUE use it.

FIG. 2.4A

`GROUP BY` collapses each department into one row. A window function keeps all seven rows and adds values computed from the partition: a row number and the gap to the previous salary within each department, sorted by salary, and the department total on every row.

RANKING · DEPARTMENTS WITH 2 OR MORE EMPLOYEES · OVER (ORDER BY N DESC)



FIG. 2.4B Six ranking functions over the same five departments, sorted by headcount n. Departments 101, 103 and 105 tie on n = 3; each function treats the tie differently.

OFFSET · DEPARTMENT 103 · OVER (PARTITION BY DEPARTMENT_ID ORDER BY SALARY DESC)

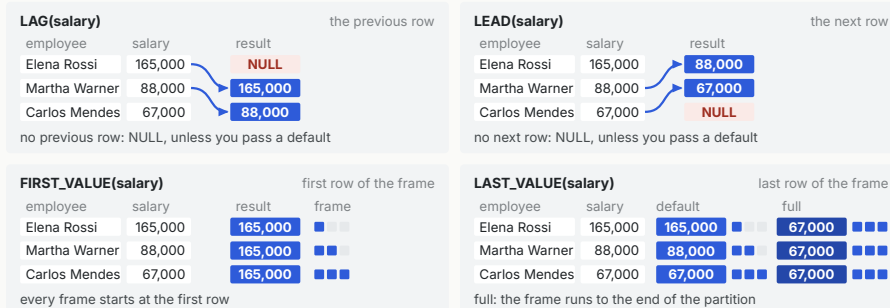


FIG. 2.4C Offset functions over department 103, sorted by salary. LAG and LEAD read a fixed distance back or forward and ignore the frame. FIRST_VALUE and LAST_VALUE read the ends of the frame; the squares show which rows each frame holds. With ORDER BY, the default frame ends at the current row, so LAST_VALUE returns the row itself; add ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING to reach the last row of the partition.

AGGREGATES · PARTITION BY DEPARTMENT_ID · THE SQUARES SHOW EACH ROW'S FRAME

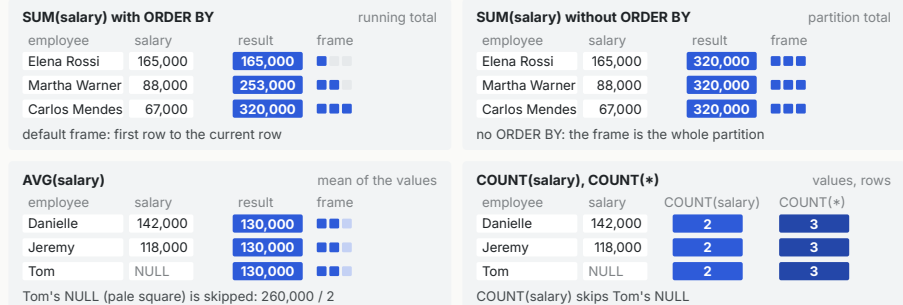


FIG. 2.4D Aggregates over a window keep every row. With ORDER BY, SUM runs over the rows up to the current one; without it, the frame is the whole partition. AVG and COUNT(salary) skip a NULL, and COUNT(*) does not.

THREE FAMILIES

Ranking functions number rows by the **ORDER BY**: **ROW_NUMBER**, **RANK**, **DENSE_RANK**, **NTILE**, **PERCENT_RANK** and **CUME_DIST**. Offset functions read another row: **LAG** and **LEAD** a fixed distance back or forward, **FIRST_VALUE** and **LAST_VALUE** the ends of the frame. Aggregates such as **SUM**, **AVG** and **COUNT** keep their usual meaning over the frame. Ranking functions, **LAG** and **LEAD** ignore the frame; aggregates and **FIRST_VALUE**, **LAST_VALUE** use it.

READING THE RESULTS

Finance sorted by salary, highest first: 138,000, 82,000 and 79,000 get row numbers 1, 2 and 3, and **salary - LAG(salary)** gives the gap to the person above: -56,000, then -3,000. The first row has no previous row, so **LAG** returns **NULL** unless you pass a default: **LEAD(hire_date, 1, DATE '9999-12-31')** gives the last hire an open end, the same shape a Type 2 dimension uses for **valid_to**. On ties, **RANK** skips (1, 2, 2, 2, 5), **DENSE_RANK** does not (1, 2, 2, 2, 3), and **ROW_NUMBER** breaks the tie in no fixed order.

WHERE THEY SIT IN A QUERY

Window functions are evaluated after **WHERE**, **GROUP BY** and **HAVING**, before the final **SELECT** list and **ORDER BY**. So a window can wrap an aggregate, as in **RANK() OVER (ORDER BY COUNT(*) DESC)** to rank departments by headcount, and a **WHERE** cannot filter on one; use **QUALIFY** or a subquery.

DIALECT DIFFERENCES

Spark supports `IGNORE NULLS` on `LAG`, `LEAD`, `FIRST_VALUE` and `LAST_VALUE`.

`LAST_VALUE(salary) IGNORE NULLS` over a running frame fills a missing salary with the last known one, which is how you forward-fill a gap. Microsoft documents `IGNORE NULLS` for SQL Server 2022 and Azure SQL, not for Fabric Warehouse, and the `NTILE` page does not list Fabric Warehouse either. Spark also accepts a named `WINDOW w AS (...)` clause; in the Warehouse, write each `OVER` in full.

WHAT IT COSTS IN SPARK

Every distinct combination of `PARTITION BY` and `ORDER BY` needs the rows shuffled and sorted. Three functions that share one `OVER` run in one `Window` operator over one `Exchange` and one `Sort`; add a function with a different `OVER` and the plan gets a second `Exchange` and `Sort`. Reuse the same window specification where the logic allows it.

RELATED CONCEPTS

Window frames: ROWS vs RANGE, Order and determinism, Logical processing order, Grouping sets and ROLLUP.

Window frames: ROWS vs RANGE.

CLAUSE FROM WHERE GROUP BY HAVING **WINDOW** SELECT ORDER BY LIMIT

A window function computes over a frame of rows around the current one. With an `ORDER BY` and no frame, both engines use `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, so rows that tie on the sort key share a result. `ROWS` moves one physical row at a time. The default is why running totals jump and why `LAST_VALUE` returns the current row.

TIES UNDER RANGE

Order the Finance department by hire date for a running payroll. Mark Lin appears twice with the same hire date, 2022-01-10. Under the default `RANGE` frame both of his rows show 299,000, because the frame takes in every row that ties with the current one. Under `ROWS` they show 217,000 and then 299,000, or 220,000 and then 299,000: which tied row comes first is not defined. Add a unique column to the `ORDER BY` when you want `ROWS` to give a repeatable answer.

`SUM(SALARY) OVER (ORDER BY HIRE_DATE ...)`

employee	hire_date	salary	RANGE (default)	ROWS
Samuel Otieno	2014-10-20	138,000	138,000	138,000
Mark Lin	2022-01-10	79,000	299,000	217,000
Mark Lin	2022-01-10	82,000	299,000	299,000

RANGE frame: up to the current row and every row tied with it

ROWS frame: up to this physical row; tie order is not defined

No ORDER BY	frame is the whole partition: the total on every row
LAST_VALUE	default frame ends at the current row: returns its own value
RANGE offsets	Spark: RANGE BETWEEN 10000 PRECEDING ... · T-SQL: ROWS only

FIG. 2.5 A running payroll for Finance, ordered by hire date. Mark Lin appears twice with the same date. For the first of his rows, the default RANGE frame takes in the tied row too and shows 299,000; ROWS stops at the current row and shows 217,000 or 220,000, depending on which tied row comes first.

FRAMES AND FUNCTIONS

Without an `ORDER BY`, the frame is the whole partition, so `SUM(salary) OVER (PARTITION BY department_id)` repeats the department total on every row. `LAST_VALUE` under the default frame stops at the current row and its ties, so it returns the current row's value; write `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` to reach the true last row. `ROW_NUMBER`, `RANK`, `LAG` and `LEAD` ignore the frame.

OFFSETS

T-SQL allows `RANGE` only with `UNBOUNDED` and `CURRENT ROW`; numeric offsets such as `3 PRECEDING` need `ROWS`. Spark also accepts value offsets under `RANGE`: `RANGE BETWEEN 10000 PRECEDING AND 10000 FOLLOWING` counts the salaries within 10,000 of the current one, and `RANGE BETWEEN INTERVAL 365 DAYS PRECEDING AND CURRENT ROW` gives a rolling year on a date column.

A WINDOW OVER EVERYTHING

A running total with no `PARTITION BY` has one partition: the whole table. Spark shows it as an `Exchange` with `SinglePartition`, and every row moves to one task to be sorted and summed. Fine for 20 rows; on a large table, partition by something or aggregate first.

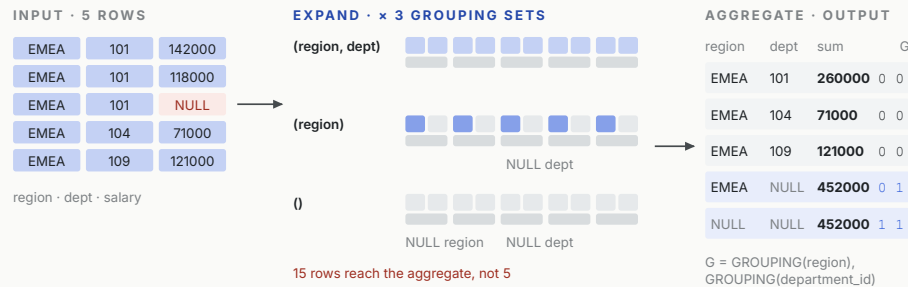
RELATED CONCEPTS

Order and determinism, Logical processing order, Grouping sets and ROLLUP.

Grouping sets and ROLLUP.

CLAUSE FROM WHERE **GROUP BY** HAVING WINDOW **SELECT** ORDER BY LIMIT

ROLLUP, **CUBE** and **GROUPING SETS** run several **GROUP BY**s in one statement and return them as one result: detail rows, subtotals and a grand total. The subtotal rows carry **NULL** in the columns they roll up, which collides with real **NULL**s unless you ask **GROUPING()** which is which.



WHICH NULL IS WHICH · GROUP BY ROLLUP (MANAGER_ID)

NULL	count 1	GROUPING = 0	a real group: the chief executive has no manager
NULL	count 20	GROUPING = 1	the grand total row added by ROLLUP

FIG. 2.6 ROLLUP (region, department_id) on the EMEA rows. Spark's Expand operator copies every input row once per grouping set, setting the rolled-up columns to NULL, then one aggregate produces details, a subtotal and a grand total. **GROUPING()** tells a rolled-up NULL from a real one.

WHAT YOU GET BACK

ROLLUP (region, department_id) is shorthand for **GROUPING SETS** ((region, department_id), (region), ()): one row per department, one per region, and one grand total. **CUBE** returns every combination, 2 to the power n grouping sets for n columns. On the sample payroll the EMEA subtotal is 452,000 and the grand total 2,001,000, which is lower than the full payroll because the inner join to departments drops the contractor in department 111.

WHICH NULL IS WHICH

Roll up by **manager_id** and two rows show **NULL**: the chief executive, whose manager really is missing, with one person, and the grand total with 20. **GROUPING(manager_id)** returns 0 on the first and 1 on the second. Spark also has **GROUPING_ID()**, one bit per column; its Microsoft Learn page does not list Fabric Warehouse, so combine **GROUPING()** calls there.

WHAT SPARK DOES WITH IT

Spark plans grouping sets with an **Expand** operator that copies every input row once per grouping set, with the rolled-up columns set to **NULL**, before a normal aggregate and shuffle. A **ROLLUP** over three columns has four grouping sets, so the aggregate reads four times the input rows; a **CUBE** over three has eight. Pre-aggregate to the finest level first when the input is large.

IN FABRIC WAREHOUSE

The Warehouse and the SQL analytics endpoint support **ROLLUP**, **CUBE**, **GROUPING SETS**, **GROUP BY ()** and **GROUP BY ALL**. The older **WITH ROLLUP** and **WITH CUBE** syntax is not part of Fabric's **GROUP BY**.

RELATED CONCEPTS

NULL and three-valued logic, Bags, sets and DISTINCT, Logical processing order.

Subqueries and decorrelation.

CLAUSE FROM WHERE **GROUP BY** HAVING WINDOW **SELECT** ORDER BY LIMIT

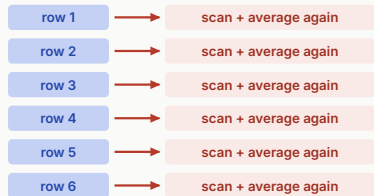
A correlated subquery reads as "run this once for every outer row". Neither engine runs it that way when it can avoid it: the optimiser rewrites the subquery as a join, with any aggregate computed once per key. Knowing the rewrite tells you what the plan will look like and which subqueries fail at run time.

WHAT SPARK TURNS IT INTO

Employees paid above their department average: **WHERE e.salary > (SELECT AVG(i.salary) FROM employees i WHERE i.department_id = e.department_id)**. In **EXPLAIN** there is no loop. There is a **HashAggregate** of **AVG** per department, joined back to employees with **salary > avg(salary)** as the join condition. **EXISTS** becomes a **LeftSemi** join, **NOT EXISTS** a **LeftAnti**, and a scalar subquery in the **SELECT** list a **LeftOuter** join to the aggregated subquery.

AS WRITTEN · ONCE PER OUTER ROW

```
WHERE e.salary > (
  SELECT AVG(i.salary) FROM employees i
  WHERE i.department_id = e.department_id)
```



20 outer rows, 20 subquery runs

WHAT RUNS · READ BOTTOM UP

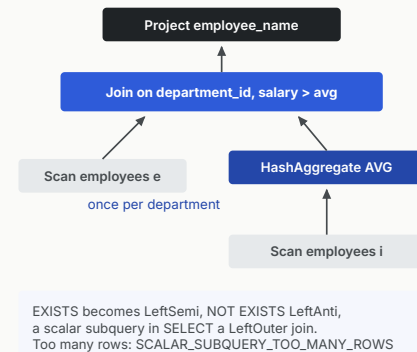


FIG. 2.7 Employees paid above their department average. Read literally, the subquery runs once per outer row. Spark's plan instead aggregates the average once per department and joins it back with salary > avg as the join condition.

SCALAR MEANS ONE ROW

A subquery used as a value must return at most one row. If it returns two, the query fails when it reaches that row: `SCALAR_SUBQUERY_TOO_MANY_ROWS` in Spark, error 512 in T-SQL, "Subquery returned more than 1 value". A duplicated key, such as employee 1007, is enough to trigger it. If it returns no rows, you get `NULL` without any error, which a later comparison then silently filters out.

WHEN THE REWRITE HURTS

A correlation on an inequality, such as counting colleagues who earn more with `WHERE i.salary > e.salary`, still decorrelates, but into a join on that inequality: every outer row is compared with every inner row. That grows with the square of the table size. For non-null salaries, a window function computes the same answer in one sorted pass: `RANK() OVER (ORDER BY salary DESC) - 1`.

RELATED CONCEPTS

Semi joins, anti joins and NOT IN, CTEs are inlined, not cached, Join cardinality and fan-out.

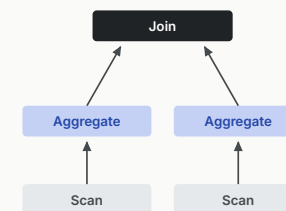
CTEs are inlined, not cached.

CLAUSE FROM WHERE GROUP BY HAVING WINDOW SELECT ORDER BY LIMIT

A `WITH` clause gives a subquery a name; it does not store the result. The engine pastes the definition into each place it is used and optimises the whole statement together. A CTE referenced three times can be computed three times.

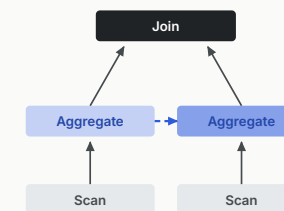
```
WITH d AS (SELECT department_id, COUNT(*) AS n FROM employees GROUP BY department_id)
SELECT * FROM d AS a JOIN d AS b ON a.department_id = b.department_id
```

FABRIC WAREHOUSE
runs d once per reference



documented: each reference runs the definition again

SPARK SQL
two branches, shuffle may be reused



ReusedExchange at run time, not a promise

MATERIALIZED
computed once, read twice

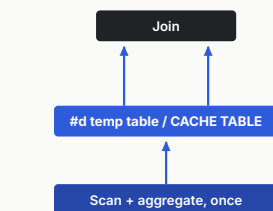


FIG. 2.8 A CTE referenced twice. Both engines paste the definition into each reference. The Warehouse runs it once per reference; Spark plans two branches and, at run time, may reuse an identical shuffle. A temp table or a cached table computes it once.

IN FABRIC WAREHOUSE

Microsoft's documentation is explicit: CTE results are not materialised, each outer reference runs the definition again, and for several references you should use a temporary object instead. In the Warehouse that is a session temp table, `CREATE TABLE #t ... WITH (DISTRIBUTION = ROUND_ROBIN)`, or a CTAS into a real table. The Warehouse supports standard, sequential and nested CTEs; nested ones are in preview.

IN SPARK AND DATABRICKS

Spark inlines a CTE when it is referenced once or when it is deterministic, so each reference becomes its own branch of the plan. At run time, adaptive execution can notice that two branches produce the same shuffle and reuse it: the final plan shows a `ReusedExchange`. That is an optimisation for identical branches, not a promise. To compute something once, cache it with `CACHE TABLE`, or write it to a table.

RANDOM VALUES

Spark does not inline a CTE that calls `rand()` or `uuid()` and is referenced more than once; it keeps one copy, so a self-join on such a CTE sees the same values on both sides. The Warehouse runs the definition once per reference, so the two sides can differ. Materialise the rows first when you need both sides to agree.

NAMING STEPS COSTS NOTHING

Because the definition is inlined, filters from the outer query reach into it: `WITH c AS (SELECT * FROM employees) SELECT * FROM c WHERE department_id = 101` scans with the filter already applied. Break a long query into named steps for readability without worrying about cost; worry only when the same expensive step is referenced more than once.

RELATED CONCEPTS

Subqueries and decorrelation, Recursive queries, Order and determinism.

Recursive queries.

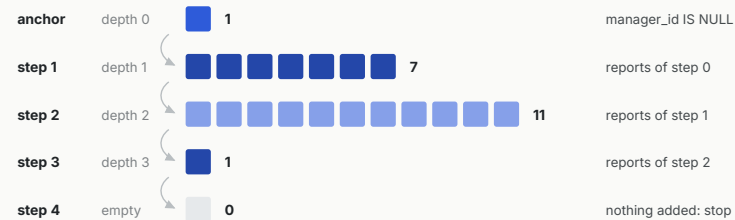
CLAUSE **FROM** WHERE GROUP BY HAVING WINDOW SELECT ORDER BY LIMIT

A recursive CTE starts from an anchor query, then repeatedly joins its own latest output until a step returns no rows. It walks hierarchies such as an org chart or a bill of materials, and generates sequences. Spark 4.1 and Databricks Runtime 17.0 support `WITH RECURSIVE`. Fabric Warehouse and the SQL analytics endpoint do not support recursive CTEs.

HOW IT ITERATES

The anchor selects the chief executive, depth 0. Each step joins employees to the rows the previous step produced: 7 direct reports at depth 1, 11 at depth 2, 1 at depth 3, then an empty step, and the query stops with all 20 rows. A step sees only the rows added by the step before it, not the whole result so far. Spark requires `UNION ALL` between the anchor and the recursive part; `UNION` fails with `UNION_NOT_SUPPORTED_IN_RECURSIVE_CTE`.

EACH STEP SEES ONLY THE ROWS ADDED BY THE STEP BEFORE



Result: 20 rows in 4 steps. Spark requires `UNION ALL` between anchor and step.

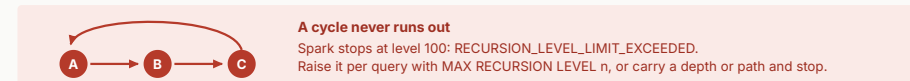


FIG. 2.9 WITH RECURSIVE over the org chart. The anchor returns the chief executive; each step joins employees to the rows the previous step added, until a step returns nothing. Spark stops at 100 levels by default, which is what a cycle in the data runs into.

LIMITS AND CYCLES

If the data contains a cycle, the steps never run out. Spark stops at 100 levels by default with `RECURSION_LEVEL_LIMIT_EXCEEDED`; raise it for one query with `WITH RECURSIVE t MAX RECURSION LEVEL 200 AS ...`. Spark 4.1 also caps the total at 1,000,000 rows through `spark.sql.cteRecursionRowLimit`. A `LIMIT` on the outer query stops generation early. To make a walk safe on dirty data, carry a depth column and stop at a sensible bound, or carry the path and stop when an id repeats.

IN FABRIC WAREHOUSE

There is no recursive CTE to fall back on. Run the walk in Spark against the same Delta table, or loop in T-SQL: insert the anchor into a temp table, then insert the next level from it in a `WHILE` loop until an insert adds no rows. For hierarchies that change rarely, compute the path or depth once during the load and store it as columns.

RELATED CONCEPTS

CTEs are inlined, not cached, Join cardinality and fan-out, Subqueries and decorrelation.

02

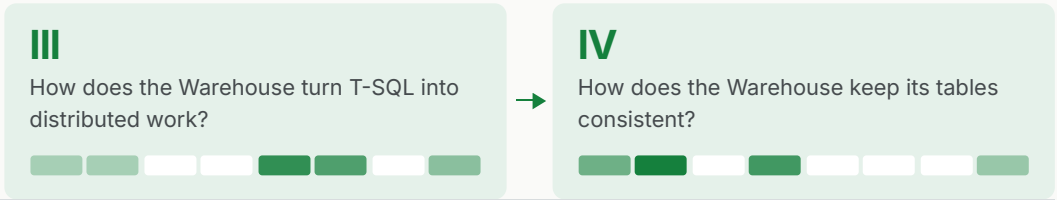
The Fabric Warehouse engine.

The Warehouse runs T-SQL differently from SQL Server: across nodes that come and go, on Parquet files in OneLake, with statistics, caches and compute pools managed for you. These two chapters cover how it plans and runs a query, then how it keeps its tables consistent: the files it writes, snapshot isolation, transactions across tables, and the history it keeps.

III How the Warehouse runs a query		
16	Warehouse architecture	21
17	Distributed plans and data movement	22
18	Statistics and cardinality estimates	23
19	Result set caching	24
20	Workload management and isolation	24
21	Reading Warehouse plans and query insights	25
IV Warehouse tables and transactions		
22	How the Warehouse writes Delta	26
23	Snapshot isolation and write conflicts	27
24	Multi-table transactions	28
25	Time travel, clones and snapshots	29

READING PATH

Layer heat: A B C D E F G H = Storage to Operate



Warehouse architecture.

LAYER **STORAGE** TABLE MODEL LOAD **QUERY** **COMPUTE** GOVERN OPERATE

Fabric Warehouse is a distributed SQL engine over OneLake. A query goes to the SQL frontend, which optimises it into a distributed plan. The distributed query processing engine (DQP) splits the plan into tasks and runs them on backend compute nodes, each reading Parquet files from OneLake through memory and SSD caches, and the results come back through the frontend. The Warehouse and the SQL analytics endpoint share this engine; only the Warehouse writes.

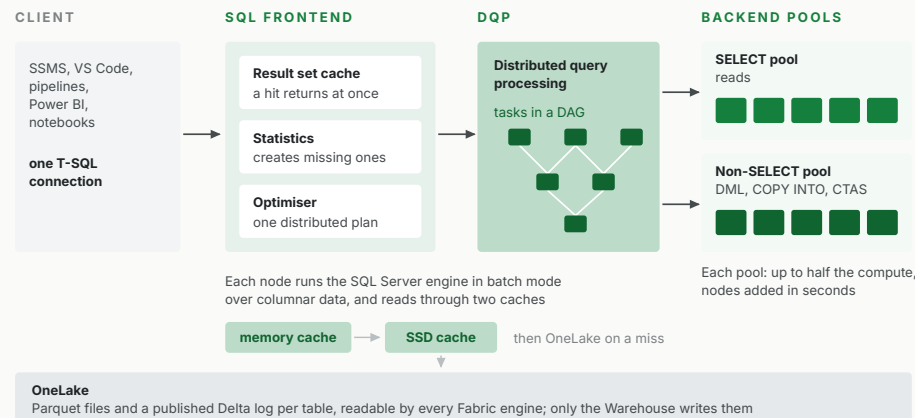


FIG. 3.1 One query through the Warehouse. The SQL frontend optimises it into a distributed plan, the DQP splits the plan into tasks, and backend nodes in one of two isolated pools run them with the SQL Server batch engine, reading through memory and SSD caches from Parquet files in OneLake. Results return through the frontend.

WHAT YOU DO NOT MANAGE

There are no indexes, no `DISTRIBUTION` option and no partitions to declare: the only storage option on a Fabric `CREATE TABLE` is `WITH (CLUSTER BY ...)`. There is no compute size to choose either. Backend nodes are added as demand rises, usually within a few seconds, and released when it falls. What you control is the data (types, clustering, file sizes), the SQL, and the capacity SKU of the workspace.

THE ENGINE ON EACH NODE

Each node runs the same engine as SQL Server, in batch mode over columnar data, so an operator works on many rows at a time and skips column segments and row groups that cannot match. Data read from OneLake is transcoded into compressed columnar structures in memory and also written to local SSD; the least recently used data is evicted first. You cannot clear these caches or turn them off.

TWO POOLS

Compute is split into a `SELECT` pool and a non-`SELECT` pool, half each, so a nightly load cannot delay the morning dashboards and a burst of dashboard queries cannot stall the load. The split, bursting and custom pools are covered under Workload management and isolation.

NOT BUILT FOR SUB-SECOND

Microsoft is explicit that the system is not optimised for consistent sub-second performance of queries that need distributed processing, because adding nodes takes seconds. Small queries run on a single node. The first run of any query can also be slow for reasons that go away on the second: statistics being created, data being cached, nodes resuming. A workspace allows 2,048 user sessions; the next one fails with `The user session limit for the workspace is 2048 and has been reached`.

COMING FROM SYNAPSE

Several habits do not carry over. There is no hash or replicated distribution to choose, and join order in the SQL does not matter: the optimiser can swap the inputs so the smaller one is the build side. Materialised views, triggers, recursive queries, synonyms, `SET TRANSACTION ISOLATION LEVEL` and manually created multi-column statistics are not supported.

RELATED CONCEPTS

Distributed plans and data movement, Workload management and isolation, How the Warehouse writes Delta, Result set caching.

Distributed plans and data movement.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

The optimiser turns a query into a plan whose steps run on many nodes, and moving rows between nodes is usually the largest cost in it. For each join it chooses between broadcasting the smaller input to every node and shuffling both inputs on the join key; steps that need every row in one place run on a single node. You steer the choice through statistics, data types and a few hints, not by declaring how a table is distributed.

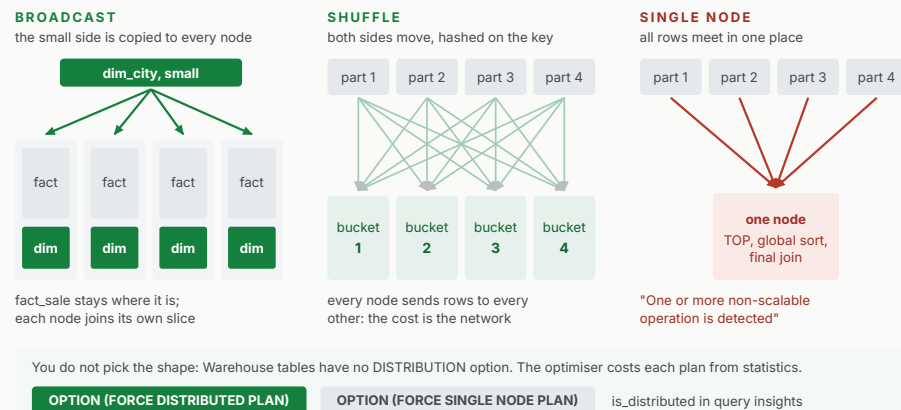


FIG. 3.2 Three shapes a Warehouse plan can take for a join. A small table is broadcast to every node; two large tables are both shuffled on the join key; steps that need all rows in one place, such as TOP or a global sort, run on a single node and can trigger the non-scalable operation warning.

BROADCAST OR SHUFFLE

When one input is small, copying it to every node is cheap and the large input stays where it is. When both are large, each row is hashed on the join key and sent to the node that owns its bucket, and the cost is the network. The optimiser costs both options from its row estimates, so the shape of the plan is only as good as the statistics: a dimension estimated at a thousand rows that really holds fifty million can be broadcast anyway.

SINGLE-NODE STEPS

TOP, a global ORDER BY, sorting the output of parallel tasks into one result and the final join all need the rows together. So does a whole plan over little data. When that step receives too many rows you get the warning `One or more non-scalable operation is detected`, and the query can run slowly or fail after a long time. Shrink what reaches that step: filter earlier and aggregate before you sort. A window with no PARTITION BY has the same problem in Spark.

FORCING THE SHAPE

If the query does not need a single node, `OPTION (FORCE DISTRIBUTED PLAN)` asks for a distributed plan; `OPTION (FORCE SINGLE NODE PLAN)` does the reverse. The Warehouse also accepts `HASH GROUP`, `ORDER GROUP`, `MERGE UNION`, `HASH UNION`, `CONCAT UNION`, `FORCE ORDER` and four `USE HINT` options that change how filter selectivity is estimated. Hints go on the top-level query only. Microsoft positions them as the last step, after statistics and schema are right.

JOIN ON MATCHING TYPES

Give columns that are joined or compared the same data type. A mismatch forces a conversion on one side, following the precedence rules from Part 1, and Microsoft's guidance is to match types and avoid explicit conversions in joins and filters. Lengths matter too: a key declared as `varchar(8000)` on one side and `varchar(20)` on the other still joins, but estimates for the wide column are less accurate.

CHECKING WHAT RAN

`SET SHOWPLAN_XML ON` returns the estimated plan without running the query. After it runs, `queryinsights.exec_requests_history` records `is_distributed: 0` means the whole query ran on one node.

RELATED CONCEPTS

Warehouse architecture, Statistics and cardinality estimates, Join cardinality and fan-out, Window frames: ROWS vs RANGE.

Statistics and cardinality estimates.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

The optimiser picks a plan by estimating how many rows each step produces, and those estimates come from statistics: histograms of column values, average lengths of long strings and table row counts. The Warehouse creates them the first time a query needs them and refreshes them when enough rows change. When they are missing or stale the estimates are wrong, and so is the plan.

WHERE THE TIME GOES

First query on a column

compile create statistics run

Same query again

compile run

After many changed rows

compile refresh statistics run

Proactive refresh moves the refresh off the query when it can.

CHANGES BEFORE A REFRESH

$\text{MIN}(500 + 0.2 \times n, \text{SQRT}(1,000 \times n))$

table rows	changed rows	share
10,000	2,500	25%
1,000,000	31,623	3.2%
100,000,000	316,228	0.32%
1,000,000,000	1,000,000	0.1%

CREATED AUTOMATICALLY

_WA_Sys_...	ACE-AverageColumnLength...	ACE-Cardinality
histogram per column DBCC SHOW_STATISTICS	varchar longer than 100 not viewable	table row count not viewable

Manual: CREATE STATISTICS ... WITH FULLSCAN on one column. Multi-column statistics are not supported.

FIG. 3.3 Statistics are created the first time a query needs them, synchronously, so that query pays for them. After enough rows change they are refreshed, on the SQL Server 2016 threshold: the share of a table that must change falls as the table grows. Three kinds are created automatically; only the histograms can be viewed.

CREATED ON FIRST USE

A column used in `WHERE`, a join, `GROUP BY`, `DISTINCT` or `ORDER BY` gets a histogram named `_WA_Sys_...` the first time a query needs one. Creation is synchronous, so that query waits for it, which is one reason first runs are slow. The engine also creates `ACE-AverageColumnLength...` for varchar columns longer than 100 and `ACE-Cardinality` for the row count; neither can be viewed. `sys.stats` with `auto_created = 1` lists them, `STATS_DATE` gives the last update and `DBCC SHOW_STATISTICS` shows a histogram.

WHEN THEY REFRESH

The Warehouse uses the SQL Server 2016 threshold: statistics are stale after $\text{MIN}(500 + 0.2 \times n, \text{SQRT}(1,000 \times n))$ changed rows. That is 31,623 changes on a table of 1,000,000 rows (3.2%) and 1,000,000 on a table of 1,000,000,000 (0.1%). Incremental refresh samples only rows added since the last refresh, which suits tables that mostly receive inserts. Proactive refresh, on by default as `PROACTIVE_STATISTICS_REFRESH`, tries to refresh `_WA_Sys_` histograms before a query needs them.

UPDATING THEM YOURSELF

After a large load with time before the first query, run `UPDATE STATISTICS dbo.t (stat_name) WITH FULLSCAN` so no report pays for the refresh. `CREATE STATISTICS ... WITH FULLSCAN` works on one column only; multi-column statistics are not supported. Two correlated filters, such as city and postcode, are estimated as if independent, and the `USE HINT` options for filter estimates are the lever the Warehouse offers.

TYPES THAT HELP THE ESTIMATE

Size strings to the data: estimates are more accurate for `varchar(100)` than for `varchar(8000)`. Lakehouse string columns defined without a length in Spark appear to the SQL analytics endpoint as `varchar(8000)`; declare `varchar(n)` in Spark SQL instead. Spark before 3.5 wrote no row-group statistics for timestamp columns, so the endpoint cannot skip row groups on them until the table is rewritten by a newer runtime.

AFTER A ROLLBACK

If a transaction inserts into an empty table, runs a `SELECT` and then rolls back, the statistics created by that `SELECT` can reflect rows that no longer exist. Update statistics on those columns after the rollback.

RELATED CONCEPTS

Distributed plans and data movement, Reading Warehouse plans and query insights, Type precedence and implicit conversion, Multi-table transactions.

Result set caching.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

Result set caching stores the final rows of an eligible `SELECT`, so the next identical query returns them without compiling or reading any data. It is on by default in every warehouse and SQL analytics endpoint, it is dropped when a referenced table changes, and it applies to a narrow class of query: the kind a dashboard repeats. Beneath it, every query reads through memory and SSD caches that are not configurable.

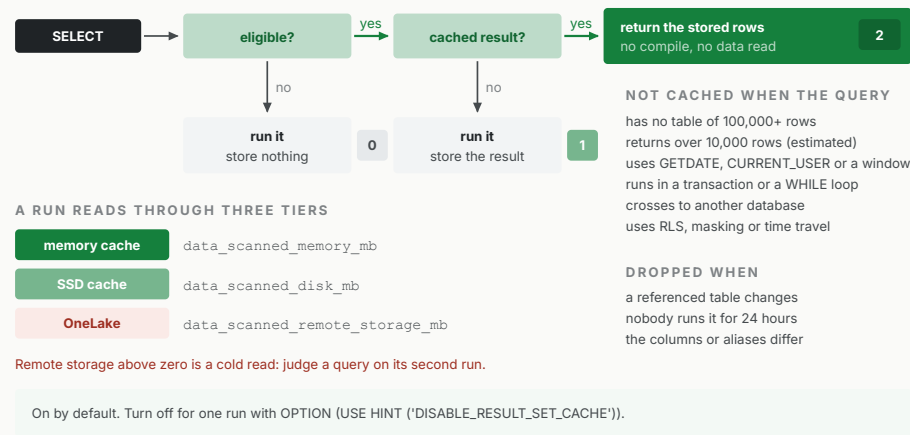


FIG. 3.4 A `SELECT` is first checked for result set caching. A hit returns the stored rows without compiling or reading data. Otherwise the query runs, reading through memory and SSD caches before OneLake, and an eligible result is stored for next time. `result_cache_hit` in query insights records which path each run took.

WHICH QUERIES QUALIFY

A pure `SELECT` that reads at least one table of 100,000 rows or more and is estimated to return 10,000 rows or fewer. It is excluded if it uses `GETDATE`, `CURRENT_USER` or another non-deterministic function, a window aggregate or `PARTITION BY ... ORDER BY`, row-level security or masking, time travel, another database, or a `varchar(max)` column in its output, and if it runs inside an explicit transaction or a `WHILE` loop. Microsoft notes these rules can change.

WHEN A STORED RESULT IS REUSED

Only for the same query with the same output columns, column order and aliases, from the same item, by a user with permission on the objects, and only if no referenced table has changed since. A result not used for 24 hours is not reused, and neither is one stored before the workspace last went offline.

CHECKING AND TURNING IT OFF

The message output says `Result set cache was used`, and `result_cache_hit` in `queryinsights.exec_requests_history` is `2` for a hit, `1` when the run stored a result and `0` when it was not eligible. Turn it off for one query with `OPTION (USE HINT ('DISABLE_RESULT_SET_CACHE'))`, or for the item with `ALTER DATABASE <item> SET RESULT_SET_CACHING OFF`.

THE CACHES UNDERNEATH

Every run reads through an in-memory cache of compressed columnar data, then local SSD, then OneLake. `data_scanned_memory_mb`, `data_scanned_disk_mb` and `data_scanned_remote_storage_mb` record where the data came from; anything read from remote storage means a cold start. The caches stay consistent with every write, and you cannot clear them.

BENCHMARKING HONESTLY

Comparing two versions of a query with caching on measures the cache. Add the `DISABLE_RESULT_SET_CACHE` hint, discard the first run, and compare runs that read nothing from remote storage.

RELATED CONCEPTS

Warehouse architecture, Reading Warehouse plans and query insights, Order and determinism, Result caches.

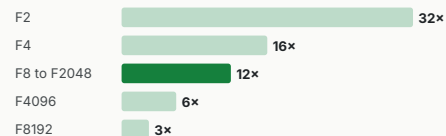
Workload management and isolation.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

The Warehouse decides how much compute each query gets. By default a workspace's compute is split into two isolated pools, one for `SELECT` and one for everything else, each scaling on demand up to half the total and each able to burst above the capacity you bought. Custom SQL pools, in preview, let a workspace admin replace that split with pools routed by application name.

DEFAULT: TWO ISOLATED POOLS

An ETL load cannot slow a dashboard, and the reverse.

Burst above the baseline, per pool

Up to 12x per pool and 24x for the workspace on most SKUs.

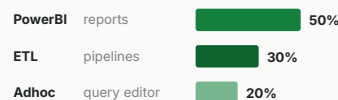
When a query cannot run within the burst limit

"This query was rejected due to current capacity constraints."

Reduce what the query reads before you raise the SKU.

CUSTOM SQL POOLS (PREVIEW)

routed by program_name, up to 8 pools; for example



Workspace admins set a maximum percentage per pool. One classifier type per workspace; a regex match to two pools picks one at random.

Watch for pressure

```
queryinsights.sql_pool_insights
is_pool_under_pressure = 1
```

FIG. 3.5 By default the workspace compute is split into a SELECT pool and a non-SELECT pool, each capped at half and each able to burst independently above the purchased baseline. Custom SQL pools, in preview, replace the split with up to eight pools routed by application name.

THE DEFAULT SPLIT

Loads, `MERGE`, `COPY INTO` and DDL run in the non-`SELECT` pool, queries in the `SELECT` pool, and neither can take the other's half, even when it sits idle. A query reaches the scheduler as a DAG of tasks, scheduled first in, first out, with a best-fit pass when there is spare capacity.

BURSTING

A pool can use more than the SKU's baseline: up to 32 times on an F2, 16 on an F4, 12 from F8 to F2048, 6 on an F4096 and 3 on an F8192. The factors apply to each pool, so a load and a report can burst at once. Burst use is borrowed from later time windows, so it is paid for afterwards. A query that cannot run even at the limit fails with `This query was rejected due to current capacity constraints`; reduce what it reads before you raise the SKU.

CUSTOM SQL POOLS

An admin can define up to eight pools per workspace, each with a maximum percentage of the backend nodes, and route requests to them by application name (128 characters at most, each name in one pool) or by a regular expression over it. A workspace uses one classifier type; if a request matches two regular expressions, the pool is picked at random. Resizing a pool leaves running queries alone, removing one cancels them.

NAME YOUR APPLICATIONS

Routing depends on the `Application Name` in each connection string, so set it in every tool and pipeline. `program_name` in `queryinsights.exec_requests_history` shows what arrived, and `DMS_user` is the Fabric portal query editor.

`queryinsights.sql_pool_insights` reports `is_pool_under_pressure` over time.

THE WORKSPACE IS THE BOUNDARY

Pools, burst limits and the 2,048-session limit are per workspace. To give a heavy read workload its own engine, Microsoft suggests OneLake shortcuts to the tables from another workspace, which serves them read-only with its own SQL compute.

RELATED CONCEPTS

Warehouse architecture, Reading Warehouse plans and query insights, Distributed plans and data movement, What a query costs.

Reading Warehouse plans and query insights.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

Three tools, by when you look. `SHOWPLAN_XML` shows the estimated plan before a query runs, the DMVs show what is running now and what blocks it, and query insights keeps finished queries for 30 days with the columns that explain their cost. Most tuning starts in query insights, because it has history and lets you find a query again by label or by shape.

BEFORE IT RUNS

`SET SHOWPLAN_XML ON`, as the only statement in its batch, makes every later statement return its plan as XML instead of running; SQL Server Management Studio draws it. Read the estimated rows at each operator, then run the query and compare the final estimate with `row_count`: a large gap points at statistics.

WHILE IT RUNS

`sys.dm_exec_requests` shows each active request with `total_elapsed_time` and `blocking_session_id`, and `sys.dm_exec_sessions` adds `login_name` and `program_name`. A workspace Admin sees every user's sessions; Members, Contributors and Viewers see their own. Only an Admin can run `KILL`, which ends the session and rolls its transaction back.

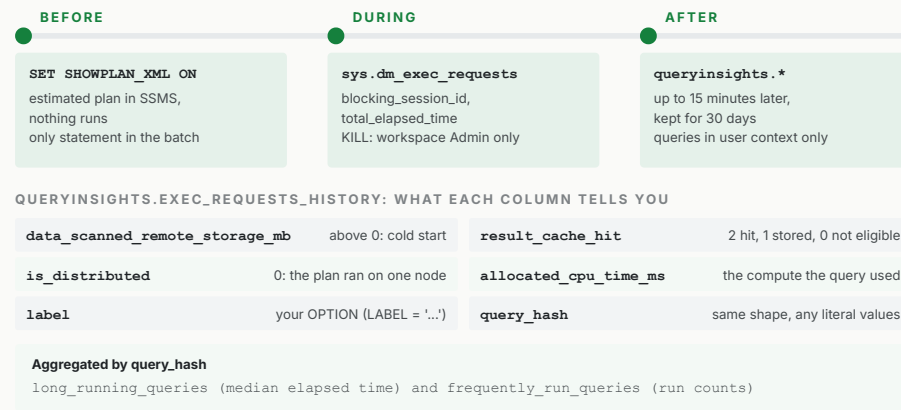


FIG. 3.6 Three places to look, by when you look. Before a query runs, SHOWPLAN_XML returns its estimated plan. While it runs, the DMVs show it and whatever blocks it. After it finishes, query insights keeps it for 30 days, usually within 15 minutes, with the columns that explain its cost.

AFTER IT FINISHES

The `queryinsights` views show user queries up to 15 minutes after they finish and keep them for 30 days, in the item you were connected to. `allocated_cpu_time_ms` is the compute used. `data_scanned_remote_storage_mb` above zero means a cold read. `result_cache_hit`, `is_distributed`, `sql_pool_name`, `status` and `error_code` explain the rest. Data scanned can understate data moved between nodes, and is 0 for `COPY INTO`.

LABEL WHAT MATTERS

Add `OPTION (LABEL = 'SALES DASHBOARD')` to a query you want to track, then filter on `label`. Queries with the same shape and different literals share a `query_hash`, and `long_running_queries` and `frequently_run_queries` aggregate on it, with `median_total_elapsed_time_ms` and `number_of_successful_runs`.

A TUNING LOOP

Take the top queries by `allocated_cpu_time_ms`. Discard cold runs. Check `is_distributed` and the non-scalable warning, then estimated against actual rows. Change one thing and compare runs of the same label.

RELATED CONCEPTS

Result set caching, Statistics and cardinality estimates, Workload management and isolation, Snapshot isolation and write conflicts.

How the Warehouse writes Delta.

Every Warehouse table is stored in OneLake as Parquet files, and the Warehouse publishes a Delta Lake log for it so other engines can read it. The Warehouse commits through its own transaction manager; a background process publishes the Delta log after the commit, and the table is read-only for every engine except the Warehouse. How you write, in a few large batches or many small ones, decides how many files the table ends up with.

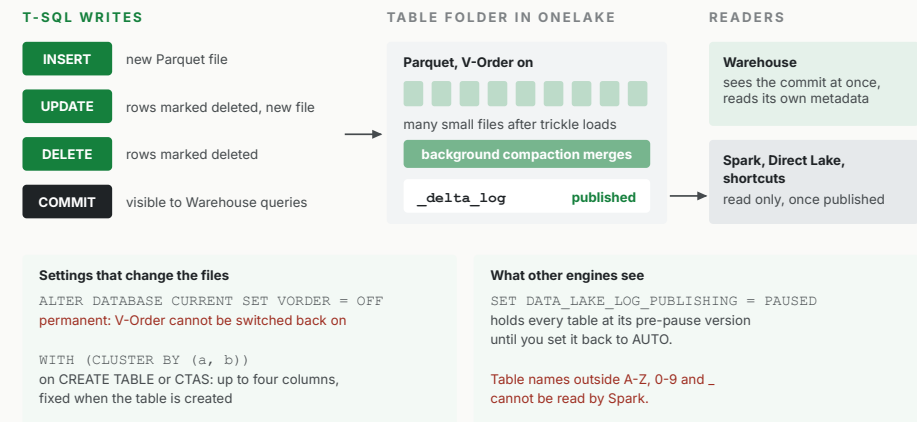


FIG. 4.1 The Warehouse writes every table as Parquet in OneLake. Each INSERT, UPDATE or DELETE writes new files; nothing is edited in place. After commit a background process publishes the Delta log, which other engines read. Background compaction merges small files and drops deleted rows.

NOTHING IS EDITED IN PLACE

Parquet files are immutable. An `INSERT` writes new files; `UPDATE` and `DELETE` mark rows as deleted and write new files for the new values. A rollback is fast because it only returns to the previous file versions. Every write adds an entry to the table's Delta log, and the Warehouse writes checkpoints for you.

SMALL WRITES MAKE SMALL FILES

Each small `INSERT`, `UPDATE` or `DELETE` creates at least one new file, so a trickle of them leaves many small files and fragmented row groups, and scans slow down until background compaction merges them and removes the deleted rows. Batch instead: stage rows and insert them in one statement, and load `COPY INTO` from files of 100 MB to 1 GB. Since October 2025 compaction backs off when it finds your locks. Lakehouse tables read through the SQL analytics endpoint are not compacted this way; run `OPTIMIZE` on them.

LAYOUT YOU CAN SET

V-Order is on by default. `ALTER DATABASE CURRENT SET VORDER = OFF` is permanent, and Microsoft suggests it only for a write-heavy staging warehouse that feeds a V-Order one; `is_vorder_enabled` in `sys.databases` shows the state. `WITH (CLUSTER BY (a, b))` on `CREATE TABLE` or `CTAS` colocates rows with similar values in up to four columns, so filters skip files. Pick mid to high cardinality columns; their order does not matter, an `IDENTITY` column cannot be one, and changing them means a new table.

WHAT OTHER ENGINES SEE

Spark, Direct Lake and anything that reads Delta see a change once its log is published, through a shortcut or the OneLake path, and cannot write. `ALTER DATABASE CURRENT SET DATA_LAKE_LOG_PUBLISHING = PAUSED` holds every table at its pre-pause version for those readers while you load; `AUTO` publishes everything since. The Warehouse itself always reads the latest data.

NAMES SPARK CAN READ

Other engines can use a table only if its name contains nothing but letters, digits and underscores, and a column only if its name has no spaces, tabs or any of `[ , ; { } ( ) = ]`. A table called `Sales Order` works in T-SQL and is unusable from a notebook.

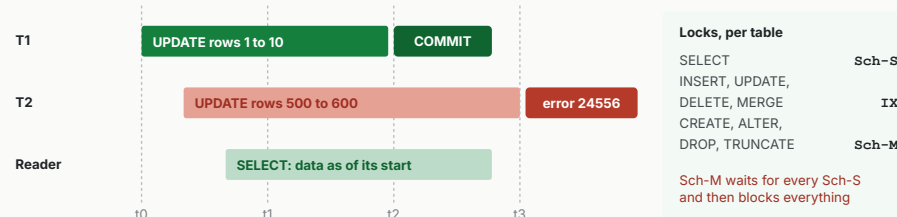
RELATED CONCEPTS

Snapshot isolation and write conflicts, Multi-table transactions, Time travel, clones and snapshots, One table, two engines.

Snapshot isolation and write conflicts.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

Every Warehouse transaction runs under snapshot isolation: it sees the data as it was when it began, readers do not block writers and writers do not block readers. Conflicts between writers are detected per table, not per row, when a transaction commits. Two updates to different rows of the same table still conflict, and the one that commits second is rolled back.

SAME TABLE, DIFFERENT ROWS

First to commit wins. The loser is rolled back whole: retry it with a delay.

TWO STATEMENTS ON ONE TABLE

INSERT + INSERT	both commit: each writes new files	UPDATE + UPDATE, any rows	second to commit fails
MERGE, even append only	can fail if another DML commits first	SELECT + any DML	neither waits for the other

FIG. 4.2 Two transactions update different rows of the same table. Both run; the first to commit wins and the second is rolled back with error 24556, because the Warehouse checks conflicts per table at commit. A reader started in between sees the table as it was when it began and never waits.

THERE IS NO OTHER LEVEL

`SET TRANSACTION ISOLATION LEVEL` is ignored and listed as unsupported, so there is no way to trade consistency for concurrency. A `SELECT` inside a transaction reads one consistent snapshot of every table it touches, however long the transaction lasts.

CONFLICTS ARE PER TABLE

The Warehouse locks whole tables, whether a statement touches one row or a million, and checks for conflicts at the end of each transaction. The first to commit succeeds. The other fails with error 24556, `Snapshot isolation transaction aborted due to update conflict`, or 24706, and everything it did is rolled back. Retry it.

WHICH STATEMENTS COLLIDE

`UPDATE`, `DELETE`, `MERGE` and `TRUNCATE` on the same table conflict. A `MERGE` that only inserts still conflicts if another write commits first. Plain `INSERT` always writes new files, so it rarely conflicts. A `SELECT` never does. Background compaction is a writer too: a transaction that inserts and then updates can lose to it.

LOCKS YOU CAN SEE

`SELECT` takes `Sch-S`, DML takes `IX`, and DDL takes `Sch-M`, which waits for every `Sch-S` and then blocks everything on the table. DDL also takes `X` locks on its rows in `sys.tables` and `sys.objects`. `sys.dm_tran_locks` with `request_status = 'WAIT'` shows who is waiting; a session that is sleeping with `open_transaction_count > 0` holds locks while doing nothing.

DESIGNING LOADS AROUND IT

Run one writer per table at a time: serialise the `UPDATE`, `DELETE` and `MERGE` jobs for a table in the orchestrator, and split parallel work by table, not by row range. Add retry with exponential backoff to every writer. Keep transactions short and schedule DDL for quiet windows.

RELATED CONCEPTS

Multi-table transactions, How the Warehouse writes Delta, Reading Warehouse plans and query insights, SCD Type 2.

Multi-table transactions.

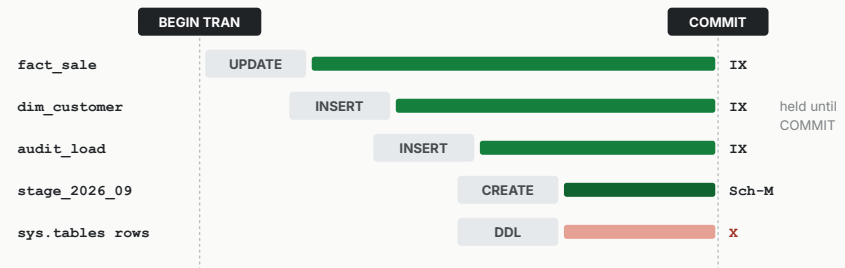
LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

An explicit transaction in the Warehouse can change several tables, including tables in other warehouses of the same workspace, and commit or roll back all of them together. DDL such as `CREATE TABLE`, `DROP TABLE` and `TRUNCATE` can run inside it. A whole load becomes atomic for Warehouse queries; other engines see it when the Delta logs are published.

WHAT CAN GO INSIDE

Besides DML: `CREATE TABLE`, `DROP TABLE`, `TRUNCATE TABLE`, `CTAS`, `sp_rename`, `ALTER TABLE` to add nullable columns, drop columns or add and drop `NOT ENFORCED` constraints, supported `ALTER COLUMN` changes, and `ALTER TABLE` on distributed temp tables. Two schema changes in one `BEGIN TRAN ... COMMIT` apply together or not at all.

`BEGIN TRAN ... COMMIT`



A failed statement or `ROLLBACK` undoes all four.

The X lock on `sys.tables` rows blocks `SELECT`s on `sys.tables`, which the Fabric portal relies on.

Also in the transaction: other warehouses in the same workspace.

Not supported: `SAVE TRANSACTION`, named or distributed transactions.

Warehouse readers: all or nothing

Spark: after the logs are published

FIG. 4.3 One explicit transaction changes three tables and creates a fourth. Locks are held on each table until `COMMIT`, and the DDL also locks its rows in the system views. At `COMMIT` every change becomes visible to Warehouse queries at once; other engines see them when the Delta logs are published.

ALL OR NOTHING

If any statement fails, the whole transaction rolls back. That is what makes the two-statement Type 2 load safe, and what lets a fact load and its dimension updates land together. In a stored procedure, wrap the work in `TRY ... CATCH` and roll back in the `CATCH`, so a failure never leaves a transaction open.

THE PRICE IS LOCKS

Every lock is held until `COMMIT`: `IX` on each table written, `Sch-M` on each table altered, and `X` on the DDL's rows in `sys.tables` and `sys.objects`. That last one blocks any `SELECT` on those catalog views, which the Fabric portal relies on, so a long transaction with DDL causes problems there too. Keep such transactions short.

ACROSS WAREHOUSES

A transaction can span warehouses in the same workspace and read from a Lakehouse's SQL analytics endpoint. Distributed transactions (`BEGIN DISTRIBUTED TRANSACTION`), savepoints, named transactions and marked transactions are not supported.

WHAT SPARK SEES

The commit is atomic inside the Warehouse. Other engines read the Delta logs, which a background process publishes after the commit. For them to keep seeing the old state until the whole load is done, set `DATA_LAKE_LOG_PUBLISHING = PAUSED` before it and `AUTO` after.

RELATED CONCEPTS

Snapshot isolation and write conflicts, SCD Type 2, How the Warehouse writes Delta, Statistics and cardinality estimates.

Time travel, clones and snapshots.

LAYER **STORAGE** **TABLE** MODEL LOAD QUERY COMPUTE GOVERN **OPERATE**

The Warehouse keeps every version of its data for a retention period, 30 days by default and anything from 1 to 120, and four features read from that history: a query hint that reads a past time, a clone that copies one table as of a time, a warehouse snapshot that exposes the whole warehouse read-only at a time, and restore points that overwrite the warehouse. None of them copies Parquet files.

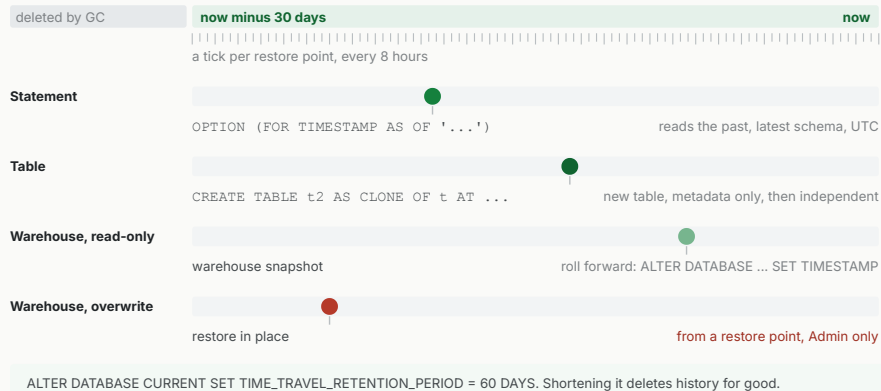
RETENTION WINDOW

FIG. 4.4 Everything inside the retention window, 30 days by default and 1 to 120 when set, can be read or recovered four ways, all from the same Parquet files: a statement reads a past time, a clone copies one table as of a time, a snapshot exposes the whole warehouse read-only, and a restore point overwrites the warehouse.

READING THE PAST IN A QUERY

`SELECT ... OPTION (FOR TIMESTAMP AS OF '2026-09-01T08:00:00.000')` applies to the whole statement, joined tables included. The time is UTC with at most three decimal places, or you get Msg 22440. The hint appears once, only in statements that begin with `SELECT`, not in a view definition, and leaves `#temp` tables alone. It returns the latest schema, so naming a column added after that time fails.

CLONING A TABLE

`CREATE TABLE dbo.t2 AS CLONE OF dbo.t AT '...'` copies metadata only and points at the same files. From then on the two tables are independent. The clone keeps row-level security, masking, object permissions, and primary and unique keys. It must be in the same warehouse, cannot go back past a schema change, and does not work on the SQL analytics endpoint. Clone related tables at one timestamp to keep them consistent.

A READ-ONLY WAREHOUSE

A warehouse snapshot is a child item fixed at one time. `ALTER DATABASE [snapshot] SET TIMESTAMP = CURRENT_TIMESTAMP` rolls it forward instantly, as one change, while running queries finish on the old version. Point reports at the snapshot and roll it forward after each load is checked. The data is frozen, not the schema: a dropped column breaks it, permissions follow the parent at once, and Power BI must use DirectQuery or Import, not Direct Lake.

RESTORING THE WAREHOUSE

Fabric creates a restore point every eight hours while the warehouse is active, an eight-hour recovery point objective, and you can add your own before a risky release. Restoring overwrites the whole warehouse under the same name and needs the Admin role. At 30 days of retention or more, at least 20 restore points are always kept.

RETENTION IS THE BUDGET

`ALTER DATABASE CURRENT SET TIME_TRAVEL_RETENTION_PERIOD = 60 DAYS` sets the window. Longer costs storage for every version of changed data, which adds up with daily full reloads. Shortening it is irreversible: history outside the new window is removed and does not come back if you lengthen it again. `time_travel_retention_cutoff_date` in `sys.databases` shows the real earliest point. Dropping and recreating a table erases its history.

RELATED CONCEPTS

How the Warehouse writes Delta, Multi-table transactions, SCD Type 2, One table, two engines.

03

Fabric SQL surfaces.

Fabric puts SQL in front of more than one item: the Warehouse, a read-only endpoint on every lakehouse, mirrored databases and SQL databases. This chapter covers which item answers which query, what the endpoint actually reads, how items join across a workspace, whose security rules apply on each path, and the ways data arrives.

V Fabric SQL surfaces		
26	Warehouse, endpoint and Lakehouse	31
27	What the SQL analytics endpoint reads	31
28	Cross-database queries	32
29	SQL permissions vs OneLake security	33
30	Direct Lake fallback	34
31	Ingestion paths	35
32	SQL database and mirroring	36

READING PATH Layer heat: A B C D E F G H = Storage to Operate

V

Which Fabric item answers which query, and who may read it?

Warehouse, endpoint and Lakehouse.

LAYER

STORAGE

TABLE

MODEL

LOAD

QUERY

COMPUTE

GOVERN

OPERATE

Fabric gives a SQL query three places to land, all on the same engine. A lakehouse is written by Spark and exposes its Delta tables read-only through a SQL analytics endpoint. A warehouse is written and read in T-SQL. Mirrored databases and SQL databases get an endpoint too. Which item holds a table decides how it is written, secured and kept.

	Lakehouse	SQL analytics endpoint	Warehouse
Writes with	Spark, pipelines	nothing: read-only	T-SQL, COPY INTO
Tables come from	Spark saveAsTable	the Delta log, synced	CREATE TABLE, CTAS
Across tables	one commit per table	no writes	one transaction
Security	OneLake roles	SQL or OneLake roles	SQL GRANT, RLS, DDM
Maintenance	you run OPTIMIZE	follows the lakehouse	automatic
History	VACUUM retention	VACUUM retention	1 to 120 days

Every lakehouse, warehouse, mirrored database and SQL database gets exactly one SQL analytics endpoint.
A workspace holds up to 150 warehouses and SQL analytics endpoints combined.

FIG. 5.1 Three Fabric items, one SQL engine. A lakehouse is written by Spark and read in T-SQL through its SQL analytics endpoint, which is read-only. A warehouse is written and read in T-SQL, with transactions across tables and maintenance done for you.

ONE ENGINE, TWO CONTRACTS

The endpoint and the Warehouse share the engine and its limitations: the same types, statistics, caches and query behaviour. The contract differs. The endpoint is read-only over tables generated from the Delta logs; you can add schemas, views, functions, procedures and permissions, but not change data. The Warehouse runs DDL and DML, with transactions across tables. Even the Spark connector can write only to a Warehouse.

WHO WRITES DECIDES

If Spark produces the table (notebooks, streaming, lakehouse pipelines), keep it in a lakehouse and read it in T-SQL through the endpoint. If SQL produces it (`MERGE`, stored procedures, `COPY INTO`), put it in a warehouse. The two mix freely: a warehouse reads lakehouse tables by three-part name and writes the results into its own tables, and Spark reads warehouse tables through their published Delta logs. Microsoft points star schemas and curated marts at the Warehouse.

UPKEEP AND HISTORY

The Warehouse compacts, checkpoints and cleans up on its own, and keeps 1 to 120 days of history. Lakehouse tables need `OPTIMIZE` and `VACUUM` from Spark or the maintenance page, and time travel through the endpoint is bounded by the `VACUUM` retention. Microsoft notes that V-Order mainly helps Direct Lake and is generally not needed for endpoint queries.

WHERE SECURITY LIVES

A warehouse uses SQL `GRANT`, row-level security, column permissions and masking. An endpoint uses either SQL permissions or OneLake security roles, depending on its access mode, and SQL rules set on an endpoint apply only to reads through that endpoint. SQL permissions vs OneLake security covers the difference.

LIMITS WORTH KNOWING

Every item has exactly one endpoint, and a workspace holds up to 150 warehouses and endpoints combined. A foreign key added on an endpoint table blocks further schema changes to it. Lakehouse strings arrive as `varchar(8000)`; `varchar(max)` is only for mirrored items and SQL databases.

RELATED CONCEPTS

What the SQL analytics endpoint reads, Cross-database queries, SQL permissions vs OneLake security, Warehouse architecture.

What the SQL analytics endpoint reads.

LAYER

STORAGE

TABLE

MODEL

LOAD

QUERY

COMPUTE

GOVERN

OPERATE

The SQL analytics endpoint stores no data of its own. A background process reads the Delta log of every table under `/Tables` and keeps a matching SQL table, with each column mapped to a T-SQL type. What you query is the lakehouse table as of the last sync, usually less than a minute behind Spark.



Lag is usually under a minute. Sync stops after 15 minutes with no queries; many lakehouses in one workspace slow it.

DELTA TO T-SQL

STRING	varchar(8000), truncated at 8 KB
VARCHAR(n), n < 2000	varchar(4 × n)
TIMESTAMP	datetime2
DOUBLE / FLOAT	float / real
TINYINT	smallint
other types	column not shown

Strings: binary UTF-8 collation, case-sensitive.

Stale data in T-SQL after a Spark write is sync lag, not a caching bug.

NOT VISIBLE TO THE ENDPOINT

anything under /Files
Delta tables outside /Tables
column mapping by ID
schemas named sys, db_owner and the like

FORCE A REFRESH

Refresh button or REST API: whole item
EXEC sys.sp_dw_refresh_ext_table 'dbo.t'
one table, new metadata sync (preview) only

FIG. 5.2 The SQL analytics endpoint does not store data. A background process reads each Delta log under /Tables and keeps a SQL table in step with it, usually within a minute. Files outside /Tables are ignored, and Delta types are mapped to T-SQL types on the way.

HOW FRESH IT IS

Under normal conditions the lag is under a minute; a large burst of changes takes longer. There is one sync process per workspace, so many lakehouses in one workspace raise the lag, and Microsoft suggests separate workspaces when that happens. The process runs while the endpoint is in use and stops after 15 minutes without queries. Tables with many small files or partitions also take longer to sync.

FORCING A REFRESH

The Refresh button in the portal, or the Refresh SQL analytics endpoint metadata REST API, re-reads the whole item; use it after schema changes. Endpoints created with the new metadata sync (preview since May 2026) also check freshness when a query arrives, accept `EXEC sys.sp_dw_refresh_ext_table 'dbo.t'` for one table, and report `last_update_time_utc` and `is_blocked` in `sys.dm_db_external_tables_log_status`.

TYPES ON THE WAY IN

STRING becomes `varchar(8000)` in a lakehouse endpoint, and longer values are truncated at 8 KB. VARCHAR(n) under 2,000 becomes `varchar(4 * n)`, TIMESTAMP becomes `datetime2`, DOUBLE becomes `float`, FLOAT becomes `real` and TINYINT becomes `smallint`. A Delta type with no mapping is simply left out of the SQL table. Strings get the binary `Latin1_General_100_BIN2_UTF8` collation, so comparisons are case-sensitive.

WHAT IT CANNOT SEE

Only Delta tables under `/Tables` appear. Files under `/Files`, and Delta tables that Spark code created at other paths, do not; a shortcut in `Tables` makes an external table visible. Column mapping by ID is not supported, and schemas named like system schemas or database roles, such as `sys` or `db_owner`, fail to sync.

STALE OR MISSING TABLES

Check, in order: the sync lag, the table's location, column mapping, unsupported types, and a foreign key on the endpoint table that blocks schema updates. In user identity mode, a security role that names a renamed column also stops the sync until it is fixed.

RELATED CONCEPTS

Warehouse, endpoint and Lakehouse, Type precedence and implicit conversion, Collation and case sensitivity, One table, two engines.

Cross-database queries.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

Inside a Fabric workspace, any warehouse or SQL analytics endpoint can be referenced from any other with a three-part name, `database.schema.table`, and joined in one T-SQL statement. No data moves between the items: they all read Delta files in OneLake. The query runs in the item you are connected to.

WHAT YOU CAN REACH

Warehouses, lakehouse endpoints, mirrored databases and SQL database endpoints in the same workspace, and their views. All of them must be in one region; connections across regions are not supported. Another workspace cannot be named in T-SQL. Spark accepts four-part names such as `workspace.lakehouse.schema.table`; in T-SQL, create a OneLake shortcut in a lakehouse and query it through that lakehouse's endpoint, which can take a few minutes to show it.

WRITING ACROSS ITEMS

Only warehouse tables accept writes. `INSERT INTO Inventory.dbo.t SELECT ... FROM Lakehouse.dbo.s` and CTAS over other items are the standard way to bring lakehouse data into a warehouse. A transaction can span several warehouses in the workspace and read from endpoints on the way.

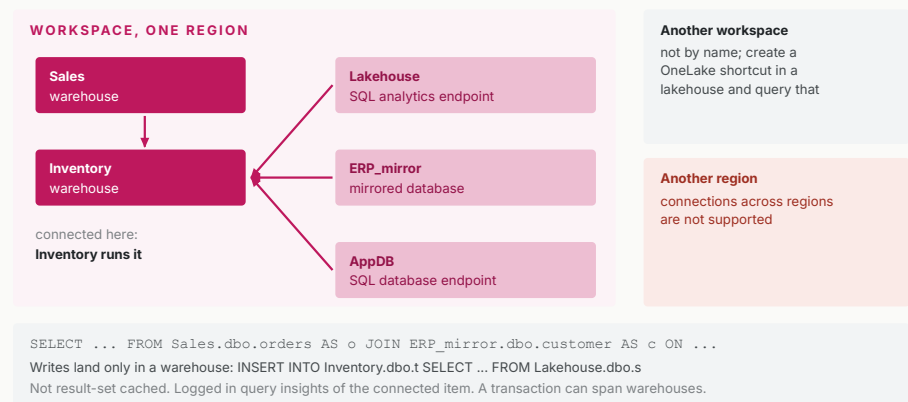


FIG. 5.3 Inside one workspace, any warehouse, SQL analytics endpoint, mirrored database or SQL database endpoint can be joined with three-part names in one T-SQL statement. The query runs in the item you are connected to. Other workspaces are reached through shortcuts; other regions not at all.

WHERE IT RUNS AND IS RECORDED

The item you are connected to runs the query, so it appears in that item's query insights. Result set caching does not apply, because the query reads objects outside the connected item.

ACCESS AND COLLATION

Workspace roles cover every item in the workspace. A user who was given one item by sharing has access to that item only, so a query naming a second item fails for them. A warehouse can be created case-insensitive while lakehouse strings are binary; when a join compares the two, choose which collation applies with `COLLATE`.

RELATED CONCEPTS

Warehouse, endpoint and Lakehouse, Multi-table transactions, Result set caching, Collation and case sensitivity.

SQL permissions vs OneLake security.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE **GOVERN** OPERATE

The same table can be read through the SQL engine or straight from its files in OneLake, and each path has its own rules. SQL permissions, row-level security and masking are enforced only inside the SQL engine. OneLake security roles are enforced on the data itself, and the SQL analytics endpoint applies them only when it runs in user identity mode.

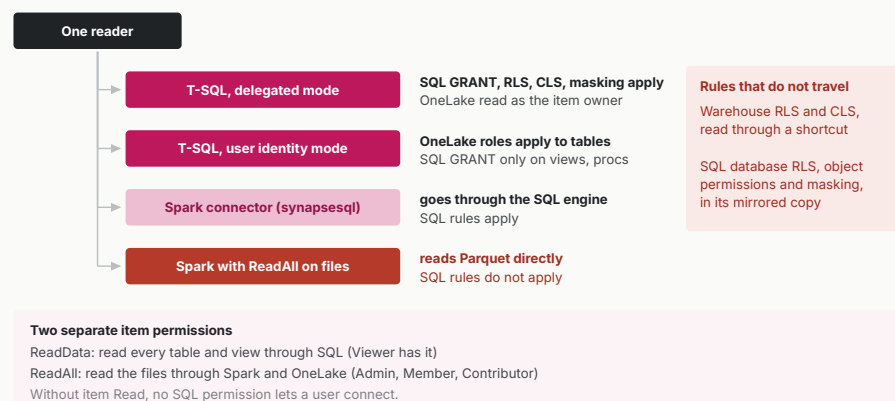


FIG. 5.4 Four ways to read the same table, and which rules apply on each. SQL rules hold only inside the SQL engine: a Spark reader with ReadAll, a shortcut to a warehouse table, or the mirrored copy of a SQL database sees the rows those rules hide. OneLake security roles follow the data when the endpoint runs in user identity mode.

TWO MODES ON THE ENDPOINT

New endpoints start in delegated mode: the SQL engine checks `GRANT`, `CREATE SECURITY POLICY`, column grants and masking, then reads OneLake as the item owner, so OneLake roles do not apply. In user identity mode the signed-in user's identity goes to OneLake, whose roles decide table, row and column access; `GRANT` on tables is not allowed, `GRANT` on views, procedures and functions still is, and masking is not supported. Switching mode cancels every running query on every endpoint in the workspace, and switching to user identity mode deletes the existing SQL roles.

ITEM PERMISSIONS UNDERNEATH

Item Read lets a user connect; without it no SQL permission helps. **ReadData** reads every table and view through SQL, and Viewers have it. **ReadAll** reads the files through Spark and OneLake, and Admins, Members and Contributors have it. The two do not overlap: a user with ReadAll opens the Parquet in a notebook and never meets SQL row-level security.

RULES THAT STAY BEHIND

Row-level, column-level and object security defined in a warehouse hold only through its SQL endpoint; read through a shortcut, the data can appear in full. A Fabric SQL database does not pass its row-level security, object permissions or masking to its mirrored copy. The Spark connector (`synapsesql`) is the exception on the Spark side: it reads through the SQL engine, so SQL rules apply.

CHOOSING WHERE RULES LIVE

If nearly every reader comes through T-SQL, keep delegated mode and write the rules in SQL. If the same data is read by Spark, Direct Lake and T-SQL, define OneLake roles and use user identity mode, so one definition applies on every path. Rules defined in SQL on an endpoint also push Direct Lake on SQL into DirectQuery.

CHANGES TAKE TIME

A OneLake role change can take up to 5 minutes to reach the endpoint, where it appears as a role with the `OLS_` prefix that you should not edit. Item permission changes can take up to two hours. In delegated mode, a change to the owner's access can take 30 to 60 minutes while a cached storage token expires.

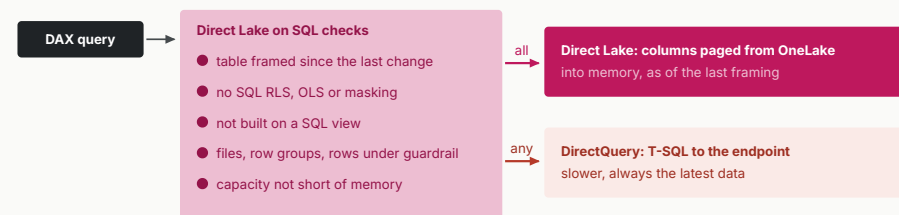
RELATED CONCEPTS

Direct Lake fallback, SQL database and mirroring, Cross-database queries, One table, two engines.

Direct Lake fallback.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

A Direct Lake semantic model answers DAX by paging columns of Delta tables from OneLake into memory, with no import refresh. Direct Lake on SQL, which finds its tables through a SQL analytics endpoint, falls back to DirectQuery, sending T-SQL to that endpoint, when a table breaks one of its conditions. By default the report keeps working, slower, and nobody is told.



DIRECTLAKEBEHAVIOR		GUARDRAILS PER TABLE		
		files	row groups	rows
Automatic	falls back silently (default)			
DirectLakeOnly	fails with an error			
DirectQueryOnly	always DirectQuery			
EVALUATE TABLETRAITS()				
[DirectLakeFallbackInfo] gives the reason per table				
One table over a guardrail takes the whole model out of Direct Lake.				

FIG. 5.5 A DAX query against Direct Lake on SQL reads columns straight from OneLake only while every condition holds. One failure, such as SQL row-level security, a view or a table over the guardrails, sends the query to the SQL analytics endpoint as DirectQuery. Direct Lake on OneLake never falls back; it errors instead.

WHAT SENDS A QUERY TO SQL

Row-level security, masking or object-level security defined in SQL on a referenced table. A model table built on a SQL view. A Delta table over a guardrail for files, row groups or rows. A table not framed since it was created or changed, or a capacity short of memory. One table over a guardrail takes the whole model out of Direct Lake.

THE GUARDRAILS

Per table: 1,000 Parquet files, 1,000 row groups and 300 million rows from F2 to F32; 5,000, 5,000 and 1,500 million on F64; 10,000, 10,000 and 12,000 million from F512, rising to 24,000 million rows from F1024. They are checked per query, except the model size limit. `OPTIMIZE` and `VACUUM` bring files and row groups down; after that, a larger SKU.

MAKING FALLBACK VISIBLE

The `DirectLakeBehavior` property decides: `Automatic`, the default, falls back silently; `DirectLakeOnly` fails with an error; `DirectQueryOnly` always uses DirectQuery, for measuring the cost. `EVALUATE TABLETRAITS()` returns `[DirectLakeFallbackInfo]` per table, where `None` means Direct Lake. Develop with `DirectLakeOnly` so a fallback shows up as an error.

DIRECT LAKE ON ONELAKE

This variant reads OneLake without going through an endpoint. It never falls back: over a guardrail its refresh fails, as an import model would. It ignores SQL row-level security and needs the user to have access to the files. Microsoft recommends it for new models.

SQL CHOICES THAT DECIDE IT

Put row-level security in the semantic model or in OneLake roles, not in SQL on the endpoint. Materialise a view as a Delta table before you model it. Keep file counts down. Relationships need matching data types and unique values on the one side; a duplicate there makes queries fail.

RELATED CONCEPTS

SQL permissions vs OneLake security, What the SQL analytics endpoint reads, Join cardinality and fan-out, How the Warehouse writes Delta.

Ingestion paths.

LAYER STORAGE TABLE MODEL **LOAD** QUERY COMPUTE GOVERN OPERATE

Data reaches a Warehouse table in a few ways: **COPY INTO** from files, T-SQL that reads other tables in the workspace, a client bulk-copy API, or a pipeline or dataflow. They all land in the same place and differ in throughput, in whose identity reads the source, and in how much they fragment the table.

COPY INTO FOR FILES

The highest-throughput path. It reads CSV, JSONL and Parquet from ADLS Gen2, Blob Storage or OneLake, where the path must use workspace and item IDs and cannot point at a warehouse. A folder means every file in it and its subfolders, except hidden files and names starting with `_` or `.`. Aim for many files of 100 MB to 1 GB, and run parallel loads into different tables. **BULK INSERT** runs as **COPY INTO**.

BAD ROWS AND PRECISION

For CSV and JSONL, **MAXERRORS** tolerates rejected rows and **ERRORFILE** writes them, with their reasons, to a `_rejectedrows` folder; neither works with Parquet. A value with more precision than its column is truncated, not rounded, for numeric, date and time types.

SOURCE	PATH	NOTES
Files: ADLS, Blob, OneLake Files	COPY INTO	CSV, JSONL, Parquet; highest throughput
Same files, older scripts	BULK INSERT	runs as COPY INTO
Spark DataFrame	<code>synapsesql write</code>	stages files, then COPY INTO
Tables in this workspace	CTAS, INSERT ... SELECT	three-part names, OPENROWSET
An application	BCP API (preview)	SqlBulkCopy, bcp.exe
Anything with a connector	pipeline, dataflow	no code, scheduled
Every path ends in a Warehouse table: the SQL analytics endpoint takes no writes.		
Size the files COPY INTO: 100 MB to 1 GB each, many files files read with OPENROWSET: at least 4 MB no singleton INSERTS		COPY INTO surprises extra precision is truncated, not rounded MAXERRORS does not work with Parquet files starting with <code>_</code> or <code>.</code> are skipped

FIG. 5.6 Ways data reaches a Warehouse table. File loads go through COPY INTO, which BULK INSERT and the Spark connector also use underneath. Tables already in the workspace arrive with CTAS or INSERT ... SELECT. Pipelines and dataflows wrap the same targets without code.

WHO READS THE SOURCE

By default the executing user's. For OneLake sources that means Contributor on both the source and target workspaces. With **CREDENTIAL = (IDENTITY = 'Workspace Identity')**, the workspace identity reads the source and the user needs only the Viewer role on the target workspace plus **INSERT** on the table. Storage accounts with public access disabled are not supported.

FROM OTHER TABLES

CTAS, **INSERT ... SELECT** and **SELECT INTO** read warehouses, endpoints and mirrored databases in the workspace by three-part name, and files through **OPENROWSET**, which is slower than a table and wants files of at least 4 MB. The Spark connector's `synapsesql write` stages the DataFrame and then runs **COPY INTO**.

WHAT TO AVOID

Singleton **INSERT** statements: each one writes files, and Microsoft's remedy for a table built that way is to rebuild it with **CTAS**. Many tiny loads have the same effect.

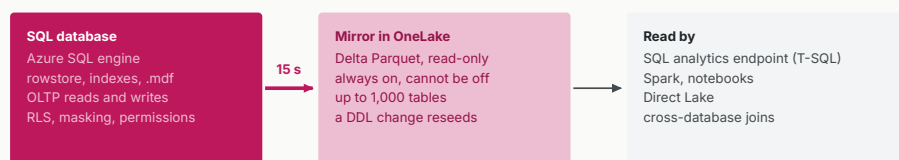
RELATED CONCEPTS

How the Warehouse writes Delta, Cross-database queries, Snapshot isolation and write conflicts, Identity and surrogate keys.

SQL database and mirroring.

LAYER **STORAGE** TABLE MODEL **LOAD** **QUERY** COMPUTE GOVERN OPERATE

SQL database in Fabric is an operational database on the Azure SQL Database engine: rowstore tables, indexes and transactions for applications. Everything it stores is mirrored automatically into OneLake as Delta, and analytics read that copy through its SQL analytics endpoint. Mirroring from other databases works the same way, from a source you configure.



LOST ON THE WAY

Security	RLS, object permissions and masking are not propagated
datetime2(7)	seventh fractional digit trimmed
LOBs	values over 1 MB truncated to 1 MB
Skipped	computed columns; xml, geography, sql_variant ...
Not mirrored	tables with json or vector columns; views, procedures

Other sources (Azure SQL, Snowflake, BigQuery ...) mirror the same way; Databricks mirrors metadata only.

FIG. 5.7 A Fabric SQL database is an Azure SQL engine for transactions. Every supported table is mirrored to Delta in OneLake, with changes published as often as every 15 seconds, and analytics read the mirror through its SQL analytics endpoint. Some types, columns and every SQL security rule do not make the trip.

TWO ENGINES, TWO COPIES

Application writes and point lookups go to the database. Analytical queries go to the mirror through the endpoint, which runs on the Warehouse engine and does not load the database. Changes are published as often as every 15 seconds, so for a few seconds the same query can return different rows on the database and on its endpoint.

MIRRORING YOU CANNOT TUNE

For a Fabric SQL database, mirroring is always on and cannot be disabled. It covers every supported table, up to 1,000 per database, into the same workspace. It pauses while the serverless database is paused and resumes where it stopped. Any DDL change on a table restarts a full snapshot of that table.

WHAT DOES NOT ARRIVE

Row-level security, object permissions and masking are not propagated: anyone who can read the mirror sees every row, so secure the endpoint again. `datetime2(7)` loses its seventh fractional digit, and a primary key of that type stops the table being mirrored. LOB values over 1 MB are truncated to 1 MB. Computed columns and types such as `xml`, `geography` and `sql_variant` are skipped, and tables with `json` or `vector` columns are not mirrored at all.

MIRRORING OTHER SOURCES

Azure SQL Database, SQL Server, Snowflake, BigQuery and others mirror the same way, with the tables you choose. The replication compute is free and uses no capacity, and mirror storage is free up to one terabyte per capacity unit, so 64 TB on an F64; queries on the mirror are charged as usual. Azure Databricks Unity Catalog is mirrored as metadata only: Fabric reads its tables through shortcuts.

RELATED CONCEPTS

SQL permissions vs OneLake security, Cross-database queries, One table, two engines, Type precedence and implicit conversion.

04

Databricks SQL.

A Databricks SQL warehouse runs the same Spark SQL as a notebook, with a layer built for SQL users around it: sizing and queueing, result caches, a query profile, procedural SQL, transactions across tables, governed metrics and model calls from a query. This chapter covers that layer and how each part compares with Fabric.

VI Databricks SQL

33	SQL warehouse sizing and scaling	38
34	Result caches	38
35	The query profile	39
36	SQL scripting and procedures	40
37	Multi-statement transactions	41
38	Metric views	41
39	AI functions in SQL	42

READING PATH

Layer heat: **A B C D E F G H** = Storage to Operate

VI

What does a Databricks SQL warehouse add around Spark SQL?



SQL warehouse sizing and scaling.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

A Databricks SQL warehouse is a set of clusters that run SQL. Its size sets the compute each cluster gives a query, from 2X-Small with one worker to 5X-Large with 512. Its cluster count sets how many queries run at once. Serverless warehouses manage the count themselves; classic and pro warehouses scale between a minimum and maximum you set.

SIZE: COMPUTE PER QUERY

2X-Small	1
X-Small	2
Small	4
Medium	8
Large	16
X-Large	32
2X-Large	64
3X-Large	128
4X-Large	256
5X-Large	512

Workers per cluster. X-Large is the default.
Spill in the profile: go up a size.

CLUSTERS: QUERIES AT ONCE

Serverless: Intelligent Workload Management
predicts each query's needs, starts it or queues it, adds clusters as queue time grows starts in 2 to 6 seconds; auto stop 10 minutes

Classic and pro: 10 queries per cluster

2 to 6 min of estimated load	+1 cluster
6 to 12 min	+2
12 to 22 min	+3
over 22 min	+3, +1 per 15 min more
a query queued 5 minutes: scale up	
low for 15 minutes: scale down; start about 4 min	

The queue holds at most 1,000 queries on any warehouse type. Watch Peak Queued Queries.

FIG. 6.1 Size and scaling answer different questions. The size sets how much compute one query gets: each step doubles the workers. The cluster count sets how many queries run at once. Serverless warehouses add clusters from predicted load and start in seconds; classic and pro follow fixed rules.

SIZE FOR THE QUERY

Each size doubles the workers of the one below, and X-Large is the default. Size makes one query faster or stops it spilling; it does not add concurrency. Increase the size when the query profile shows spill to disk.

CLUSTERS FOR CONCURRENCY

Classic and pro warehouses allow 10 concurrent queries per cluster. Autoscaling adds clusters from the estimated time to process every running and queued query: one for 2 to 6 minutes of load, two for 6 to 12, three for 12 to 22, and one more for every 15 minutes beyond that. A query queued for 5 minutes also scales the warehouse up; 15 minutes of low load scales it down. The default is one cluster, minimum and maximum.

SERVERLESS

Intelligent Workload Management predicts each query's needs, starts it if there is capacity and queues it if not, adds clusters as queue times grow and releases them when demand drops. Databricks advises starting serverless warehouses large and letting this manage concurrency. A serverless warehouse starts in 2 to 6 seconds; classic and pro take about 4 minutes. Photon runs on all three types, Predictive IO on serverless and pro, Workload Management only on serverless. Databricks recommends pro where serverless is not available or where custom networking must reach private databases.

THE QUEUE AND AUTO STOP

Any warehouse queues at most 1,000 queries; watch Peak Queued Queries. Auto stop defaults to 10 minutes on serverless (5 at least, in the UI) and 45 minutes on pro and classic (10 at least). A stopped warehouse loses its local caches.

AGAINST FABRIC

A Fabric Warehouse has no size to choose: the capacity SKU sets the burst limit and the pools split reads from writes. On Databricks isolation comes from separate warehouses, one for loads and one for reports.

RELATED CONCEPTS

Result caches, The query profile, Workload management and isolation, What a query costs.

Result caches.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

Databricks SQL keeps query results so that a repeated query does not run again: a local cache on each cluster and, on serverless warehouses, a remote cache shared across the workspace. Underneath, a disk cache keeps data files on local SSD. An update to a referenced table invalidates the stored results.

LOCAL RESULT CACHE

An in-memory cache per cluster for queries through SQL warehouses. It lasts as long as the cluster, or until it is full, and is cleared when the warehouse stops or restarts.

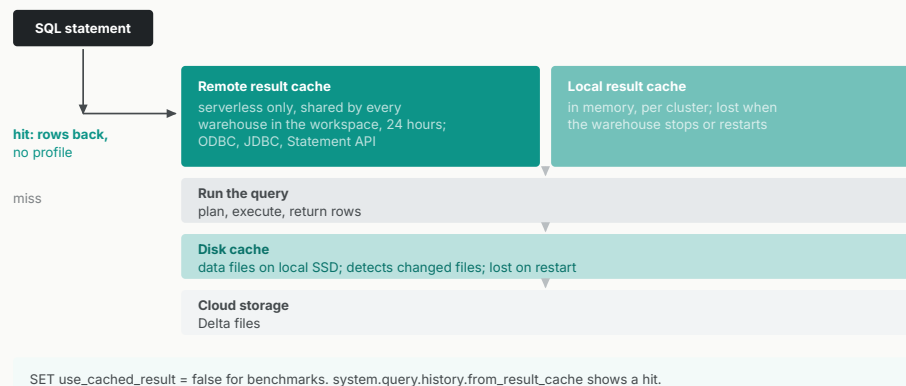


FIG. 6.2 A Databricks SQL query checks the result caches before it runs: the remote cache shared by every serverless warehouse in the workspace and the local cache of the cluster. A miss runs the query, reading data files through the disk cache on local SSD. An update to a referenced table invalidates the stored results.

REMOTE RESULT CACHE

Serverless only. It persists results as workspace system data, is shared by every warehouse in the workspace, and survives a warehouse stopping or restarting. An entry lasts 24 hours from when it was stored. It serves queries from ODBC and JDBC clients and the SQL Statement API.

DISK CACHE AND UI CACHE

The disk cache keeps copies of data files on the cluster's SSD, notices when files change, and is cleared with the cluster; it speeds up reads, not the query plan. The SQL editor also keeps each user's results and visualisations for at most 7 days.

MEASURING HONESTLY

Run `SET use_cached_result = false` before benchmarking. In `system.query.history`, `from_result_cache` is true for a cache hit and `read_io_cache_percent` shows how much was read from the disk cache. A result served from cache has no query profile. Stopping the warehouse clears the local caches, not the remote one.

AGAINST FABRIC

Fabric's result set cache is per item and has eligibility rules: a table of at least 100,000 rows, a result of at most 10,000, no non-deterministic functions. Both engines drop a stored result when a referenced table changes.

RELATED CONCEPTS

SQL warehouse sizing and scaling, The query profile, Result set caching, Order and determinism.

The query profile.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

Every statement on a SQL warehouse leaves two records: a row in `system.query.history` that says where the wall-clock time went, and a query profile that shows the plan as a graph of operators with the rows, time and memory of each. The history tells you which phase to look at; the profile tells you which step.

WALL CLOCK



QUERY PROFILE

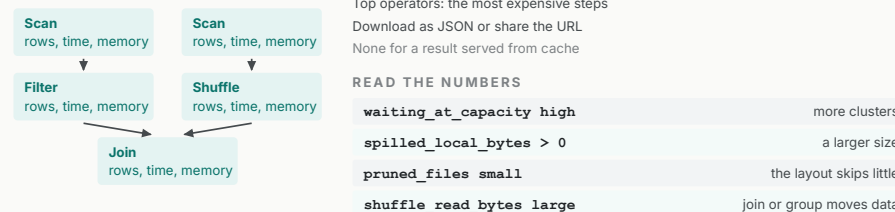


FIG. 6.3 Where a statement's time went, from `system.query.history`, and what the profile shows beneath it. Queue time points at clusters, compile time at metadata and pruning, execution at the plan; the profile's operators, rows, time and memory say which step.

WHERE THE TIME WENT

`waiting_for_compute_duration_ms` is provisioning, `waiting_at_capacity_duration_ms` the queue, `compilation_duration_ms` loading metadata and optimising, `execution_duration_ms` running, and `result_fetch_duration_ms` returning rows. `total_duration_ms` excludes the fetch. `total_task_duration_ms` sums every task, so comparing it with execution time shows how parallel the work was.

READING THE PHASES

Queue time means too few clusters. Provisioning time means a cold warehouse. Long compilation can point at metadata, such as a table with a great many files. A long fetch means too many rows sent to the client. Execution time is where the profile comes in.

THE PROFILE

A directed graph of operators, such as Scan, Filter, Shuffle and Join, each with Time spent, Memory peak and Rows. The Top operators tab lists the most expensive steps. Share its URL or download it as JSON. A query answered from the result cache has no profile.

NUMBERS THAT POINT AT A FIX

`spilled_local_bytes` above zero: a larger size. Few `pruned_files` against many `read_files`: the layout lets the query skip little. A large `shuffle_read_bytes`: a join or aggregation moving data.

AGAINST FABRIC

Fabric documents an estimated plan through `SHOWPLAN_XML` and per-query totals in query insights. The Databricks profile shows actual rows and time for each operator.

RELATED CONCEPTS

SQL warehouse sizing and scaling, Result caches, Reading Warehouse plans and query insights, Join cardinality and fan-out.

SQL scripting and procedures.

LAYER STORAGE TABLE MODEL **LOAD** QUERY COMPUTE GOVERN OPERATE

Databricks SQL runs procedural SQL. A script is one `BEGIN ... END` compound statement with variables, conditions, loops and error handlers, and a stored procedure saves a script in Unity Catalog to run with `CALL`. T-SQL has had all of this for decades. When code moves between them, what matters is what commits when and whose permissions a procedure runs with.

```
BEGIN
  DECLARE n INT DEFAULT 0;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN ... log and stop ... END;
  WHILE n < 3 DO
    INSERT INTO t SELECT ...; -- commits
    SET n = n + 1;
  END WHILE;
  MERGE INTO dim USING ...; -- commits
END;
```

Not a transaction by default

Each INSERT and MERGE is its own commit. `BEGIN ATOMIC ... END` makes the block one transaction, but allows no handlers inside.

Stored procedure

```
CREATE PROCEDURE cat.sch.load_dim
  (IN d DATE, OUT n INT)
  LANGUAGE SQL SQL SECURITY INVOKER
  AS BEGIN ... END;
CALL cat.sch.load_dim(...)
```

VERSIONS

scripting

DBR 16.3

procedures

DBR 17.0

cursors, CONTINUE

DBR 18.1

T-SQL has had all of this for decades; the difference is where it runs and what commits when.

FIG. 6.4 A SQL script is one compound statement with variables, loops and handlers. Without `ATOMIC` each statement inside commits on its own, so a failure halfway leaves the earlier ones in place. A procedure stores a script in Unity Catalog, runs as the caller or the owner, and is called with `CALL`.

HOW A SCRIPT IS LAID OUT

Declarations come first, in order: variables and conditions, then cursors, then handlers, then the statements. Before Databricks Runtime 17.2 each `DECLARE` holds one variable. Control flow covers `IF`, `CASE`, `WHILE`, `REPEAT`, `LOOP`, `FOR`, `LEAVE`, `ITERATE`, `SIGNAL`, `RESIGNAL` and `GET DIAGNOSTICS`. Scripting needs Runtime 16.3; cursors need 18.1.

HANDLING ERRORS

`DECLARE EXIT HANDLER FOR SQLEXCEPTION`, `NOT FOUND`, a `SQLSTATE` or a named condition runs the handler and leaves the compound statement. `CONTINUE` handlers, which carry on afterwards, need Runtime 18.1. The T-SQL equivalent is `TRY ... CATCH`.

WHAT COMMITS

In a plain script each `INSERT`, `UPDATE` or `MERGE` commits on its own, and a handler does not undo the statements before the failure. `BEGIN ATOMIC ... END` makes the block one transaction that commits or rolls back as a whole, but it allows no handlers and only DML, `SELECT`, variables, control flow and calls to atomic procedures. T-SQL also commits statement by statement until you open `BEGIN TRAN`.

STORED PROCEDURES

`CREATE PROCEDURE` needs Runtime 17.0 and Unity Catalog. Parameters are `IN` (the default), `OUT`, which starts as `NULL`, or `INOUT`. `LANGUAGE SQL` and one of `SQL SECURITY INVOKER` or `DEFINER` are required: the first runs with the caller's permissions, the second with the owner's. Run it with `CALL`.

RELATED CONCEPTS

Multi-statement transactions, Multi-table transactions, SQL in CI/CD, Parameters and SQL injection.

Multi-statement transactions.

LAYER

STORAGE

TABLE

MODEL

LOAD

QUERY

COMPUTE

GOVERN

OPERATE

Since July 2026, Databricks can group SQL statements across several tables into one atomic transaction, generally available. It comes in two forms: `BEGIN ATOMIC ... END`, which commits or rolls back by itself, and interactive `BEGIN TRANSACTION ... COMMIT` on SQL warehouses. Every table written must be a Unity Catalog managed table with catalog commits enabled.

NON-INTERACTIVE		INTERACTIVE	
<pre>BEGIN ATOMIC UPDATE ...; INSERT ...; END;</pre>		<pre>BEGIN TRANSACTION; UPDATE ...; INSERT ...; COMMIT;</pre>	
commit	automatic, or roll back on error	commit	you COMMIT or ROLLBACK
runs on	SQL warehouse, serverless, DBR 18.0+	runs on	SQL warehouses only
conflicts	row level	conflicts	table level, except plain appends
LIMITS			
100	100	100	48 h
tables read or written	views read	commits per table inside	then rolled back
			10 min
			idle interactive: rolled back
No DDL inside: CREATE, ALTER and DROP TABLE fail. Fabric Warehouse allows DDL in a transaction, conflicts per table, and snapshots at BEGIN; Databricks snapshots each table the first time the transaction touches it.			

FIG. 6.5 Two forms of multi-statement transaction on Databricks, generally available since July 2026. Both need Unity Catalog managed tables with catalog commits, detect conflicts at commit, and read each table from a snapshot taken at first access. They differ in where they run and how finely they detect conflicts.

TWO FORMS

`BEGIN ATOMIC` runs on any SQL warehouse, on serverless compute and on clusters from Runtime 18.0, and detects conflicts at row level. The interactive form runs on SQL warehouses only, leaves `COMMIT` and `ROLLBACK` to you, detects conflicts at table level except for unconditional appends, and rolls back after 10 minutes of inactivity.

HOW IT ISOLATES

Optimistic concurrency: nothing is locked, and at commit Databricks checks whether another transaction changed the same data since yours began; if so, yours fails. Each table is read from a snapshot taken the first time the transaction touches it, and every later read of that table uses the same snapshot.

LIMITS

Up to 100 tables read or written and 100 views read, up to 100 intermediate commits per table, and an automatic rollback after 48 hours. DDL such as `CREATE`, `ALTER` and `DROP TABLE` cannot run inside.

AGAINST FABRIC WAREHOUSE

Fabric takes table locks you can see in `sys.dm_tran_locks`, fails the second writer to a table even when the rows differ, allows DDL inside a transaction and reads everything as of `BEGIN`. A two-statement Type 2 load needs a transaction on both engines; on Databricks, `BEGIN ATOMIC` gives the finer conflict detection.

BEFORE THIS EXISTED

One `MERGE` was the unit of atomicity on Delta, and a multi-table load stayed consistent through ordering and loads that could safely run twice. That design still suits pipelines that cannot use managed tables.

RELATED CONCEPTS

Multi-table transactions, Snapshot isolation and write conflicts, SCD Type 2, SQL scripting and procedures.

Metric views.

LAYER

STORAGE

TABLE

MODEL

LOAD

QUERY

COMPUTE

GOVERN

OPERATE

A metric view stores measures as formulas in Unity Catalog, separate from the fields you group them by, so one definition answers at any grain. Queries wrap each measure in `MEASURE()` and group by the fields they need, and the engine aggregates from the base rows every time. It removes the most common reporting error in SQL: aggregating numbers that were already aggregated.

AMER, DEPARTMENT AVERAGES

103 Sales	3 salaries	106,667
105 Finance	3 salaries	99,667
110 Executive Office	1 salary	310,000
AVG of the three averages		172,111
AVG over the 7 salaries		132,714

The same trap catches ratios and distinct counts: store the formula, not a pre-aggregated number.

```
SELECT region, MEASURE(avg_salary) FROM payroll_mv GROUP BY region;
```

AMER returns 132,714. Group by department instead and the same measure returns each department's average. MEASURE() is required for every measure; SELECT * is not allowed; join a metric view through a CTE.

METRIC VIEW

```
CREATE VIEW payroll_mv
WITH METRICS LANGUAGE YAML AS $$
version: 1.1
source: hr.employees
joins:
  - name: d
    source: hr.departments
    on: source.department_id = d.department_id
fields:
  - name: region
    expr: d.region
measures:
  - name: avg_salary
    expr: AVG(salary)
$$
```

FIG. 6.6 An average of department averages is not the region's average. In the dataset, AMER's three department averages average to 172,111, because the one-person executive office counts as much as a team; the true average of its seven salaries is 132,714. A metric view stores `AVG(salary)` once and evaluates it at whatever grouping the query asks for.

THE ERROR IT PREVENTS

In the dataset, AMER's three departments average 106,667 over three salaries, 99,667 over three and 310,000 over one. The average of those averages is 172,111, because the one-person executive office counts as much as a team. The average of the seven salaries is 132,714. Ratios, distinct counts and percentages fail the same way. A view with `GROUP BY department_id` fixes the grain; a metric view does not.

DEFINING ONE

`CREATE VIEW payroll_mv WITH METRICS LANGUAGE YAML AS $$ ... $$`. In YAML version 1.1 the definition names a `source`, an optional `filter`, `joins`, the fields to group by and the `measures`, each a name and an aggregate expression such as `AVG(salary)`. Creating one needs Runtime 16.4, `SELECT` on the sources and `CREATE TABLE` in the schema.

QUERYING ONE

`SELECT region, MEASURE(avg_salary) FROM payroll_mv GROUP BY region` returns 132,714 for AMER. Every measure needs `MEASURE()`, so `SELECT *` is not allowed. A metric view cannot be joined directly: query it in a CTE and join the CTE. From Runtime 18.1, `MEASURE(m) FILTER (WHERE ...)` restricts one measure.

WHERE IT IS USED

SQL editors, notebooks, dashboards, Genie and alerts, and Power BI, Tableau and Sigma. Materialisation pre-computes aggregations, refreshes them incrementally and lets the engine rewrite queries to use them.

IN FABRIC

The same layer is the Power BI semantic model and its DAX measures, often over Direct Lake. T-SQL has no measure object; a view fixes its grain. Keep each metric in one place.

RELATED CONCEPTS

Grouping sets and ROLLUP, Bags, sets and DISTINCT, Direct Lake fallback, Join cardinality and fan-out.

AI functions in SQL.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

AI functions call a language model from inside a query, once per row: classify a ticket, extract fields from an address, summarise a review. Databricks SQL and Fabric Warehouse both offer task functions with nearly the same names, and Databricks adds `ai_query` for any served model. Each call adds latency and cost, and the answer can differ between runs.

THE TWO SETS

Both have sentiment, classify, extract, summarise, translate and grammar functions: `ai_classify` on Databricks, `AI_CLASSIFY` in Fabric. For a free prompt, Databricks has `ai_gen` and `ai_query`, Fabric `AI_GENERATE_RESPONSE`. Databricks also has `ai_mask`, `ai_similarity`, `ai_parse_document`, `ai_forecast` and `vector_search`.

ON DATABRICKS

Not available on classic SQL warehouses, and Runtime 15.4 LTS or later is required. The functions parallelise and retry for you, so send the whole table in one query. Task functions and `ai_query` on Databricks-hosted models add inference cost on top of compute. `ai_query(..., failOnError => false)` returns a struct with the response and an error message per row instead of failing the query.

TASK	DATABRICKS SQL	FABRIC WAREHOUSE (PREVIEW)
Sentiment	ai_analyze_sentiment	AI_ANALYZE_SENTIMENT
Classify	ai_classify	AI_CLASSIFY
Extract fields	ai_extract	AI_EXTRACT
Summarise	ai_summarize	AI_SUMMARIZE
Translate	ai_translate	AI_TRANSLATE
Fix grammar	ai_fix_grammar	AI_FIX_GRAMMAR
Free prompt	ai_gen, ai_query	AI_GENERATE_RESPONSE
Documents	ai_parse_document	none

Databricks

not on classic SQL warehouses
parallelism and retries handled: send the whole table in one query
ai_query(..., failOnError ⇒ false) returns the error per row instead of failing

Fabric Warehouse

typically 20 to 100 rows per second
NULL when the model cannot answer
input up to 99 KB of text
precompute into a column, do not call it in every report query

FIG. 6.7 An AI function is a model call per row, made from inside a query. Both engines offer task functions with nearly the same names; Databricks adds ai_query for any served model. Each call costs time and money, so run it once and store the result.

IN FABRIC WAREHOUSE

In preview. It needs the tenant setting for Azure OpenAI features, a paid capacity from F2 and a supported region. A function returns `NULL` when the model cannot answer, for example over 99 KB of input. Microsoft quotes 20 to 100 rows per second. Queries that call them show `is_using_external_api` in query insights.

RUN ONCE, STORE THE ANSWER

At 20 to 100 rows per second, 10,000 rows take between 100 seconds and about 8 minutes. Store the output in a column keyed to the input row and recompute only rows that are new or changed. Do not put an AI function in a view behind a dashboard.

CHECK THE OUTPUT

The result is not deterministic, like `uuid()`. Give classification a closed list of labels, validate extracted fields as you would any input, and count the `NULL`s and errors after each run.

RELATED CONCEPTS

Order and determinism, Result set caching, What a query costs, Testing SQL and data quality.

05

Working across both engines.

The same dimension table is often loaded by one engine and read by the other. This part covers the modelling that has to hold on both: keys neither engine enforces, surrogate keys generated in a distributed system, and slowly changing dimensions written with MERGE, then the operational side of running SQL across the two.

VII Modelling across both engines

40	Constraints the optimiser trusts	45
41	Identity and surrogate keys	45
42	SCD Type 1	46
43	SCD Type 2	47

VIII Operating across both engines

44	One table, two engines	48
45	Parameters and SQL injection	49
46	Testing SQL and data quality	50
47	SQL in CI/CD	50
48	What a query costs	51

READING PATH

Layer heat: A B C D E F G H = Storage to Operate

VII

How do keys and history work when either engine loads the table?



VIII

What breaks when the same data meets both engines?



Constraints the optimiser trusts.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

Both engines let you declare primary and foreign keys, and neither checks them. Fabric Warehouse accepts them only as `NOT ENFORCED` and documents no use of them in query planning. Databricks can be told to rely on a key, and then rewrites queries on the assumption that it holds. Constraints that are enforced, `NOT NULL` and, on Delta, `CHECK`, stop bad rows when they are written.

CONSTRAINT	FABRIC WAREHOUSE	DATABRICKS
PRIMARY KEY, UNIQUE	NONCLUSTERED NOT ENFORCED	informational; RELY lets it optimise
FOREIGN KEY	NOT ENFORCED	informational; RELY from 15.4
NOT NULL	enforced	enforced: the write fails
CHECK	not among the supported constraints	enforced: the write fails

DEPARTMENTS WITH 105 LOADED TWICE, AND A PRIMARY KEY (DEPARTMENT_ID) RELY

```
SELECT DISTINCT department_id FROM departments
```

as written: 10 rows

RELY may drop the DISTINCT: 11 rows

```
SELECT SUM(salary) FROM employees e LEFT JOIN departments d USING (department_id)
```

as written: 2,360,000 over 23 rows

RELY may drop the join: 2,061,000

RELY is a promise you keep with a duplicate check after every load. The optimisations need Photon.

FIG. 7.1 Neither engine enforces a primary key. Databricks can be told to rely on one, and then removes work the key makes redundant: a `DISTINCT` over the key, a `LEFT JOIN` that cannot add rows. If the data breaks the key, the rewritten query answers differently from the one you wrote. Here department 105 has been loaded twice.

IN FABRIC WAREHOUSE

The documented table constraints are `PRIMARY KEY NONCLUSTERED NOT ENFORCED`, `UNIQUE NONCLUSTERED NOT ENFORCED` and `FOREIGN KEY ... NOT ENFORCED`, added with `ALTER TABLE`, never inline in `CREATE TABLE`. `DEFAULT` constraints are not supported. A declared key tells readers and tools what should be true; nothing in the engine checks it.

IN DATABRICKS

`NOT NULL` and `CHECK` are enforced: a violating write fails with an error. Primary and foreign keys on Unity Catalog tables are informational, generally available from Runtime 15.2; `UNIQUE` is in preview from 18.2. Add `RELY` (primary keys from 14.2, foreign keys from 15.4) and the engine may drop a `DISTINCT` over the key, or a `LEFT JOIN` to it that cannot add rows. These rewrites need Photon, which SQL warehouses and serverless compute run by default.

WHEN THE PROMISE BREAKS

Load department 105 twice and keep the `RELY` key. `SELECT DISTINCT department_id FROM departments` means 10 rows, but with the `DISTINCT` dropped it returns 11. `SUM(salary) over employees LEFT JOIN departments` means 2,360,000 over 23 rows, but with the join dropped it returns 2,061,000. Databricks puts it plainly: relying on a constraint that is not satisfied can lead to incorrect results.

KEEP THE PROMISE

Before you declare `RELY`, and after every load, run the checks that prove it: `GROUP BY key HAVING COUNT(*) > 1` must return nothing, and so must a `NOT EXISTS` search for orphaned foreign keys. In Fabric the same checks protect the Power BI relationships built on those keys, whose one side must be unique.

RELATED CONCEPTS

Bags, sets and `DISTINCT`, Join cardinality and fan-out, Testing SQL and data quality, Identity and surrogate keys.

Identity and surrogate keys.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

A surrogate key is a meaningless value that identifies a dimension row independently of its business key, so a Type 2 dimension can hold several versions of one employee. Both engines generate them with identity columns that promise uniqueness and nothing else: not order, not contiguity. A hash of the business key is the alternative that gives the same value everywhere.

Fabric IDENTITY	Delta identity column	Hash of the business key
BIGINT IDENTITY no seed, no increment unique; not in order; gaps IDENTITY_INSERT for -1 unknown, then DBCC CHECKIDENT RESEED cannot be added with ALTER TABLE	GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1) unique; not contiguous BY DEFAULT accepts explicit values disables concurrent transactions cannot be added with ALTER TABLE	xxhash64(employee_id) or sha2(...): same input, same key stable across reloads and engines no sequence to coordinate wider than an integer; collisions possible with 64 bits

THE SAME FOUR NEW ROWS, TWO LOADS

employee_id	1019	1020	1021	1022	Key values carry no meaning: never sort or filter on them.
IDENTITY, for example	88	89	2011	2012	
hash key	h(1019)	h(1020)	h(1021)	h(1022)	

Each load gets its own range;
gaps between loads are normal.

FIG. 7.2 Three ways to give a dimension row its own key. Identity columns on both engines are unique but not ordered and have gaps; Delta identity also turns off concurrent writes to the table. A hash of the business key is the same on every load and on both engines, at the price of a wider key.

FABRIC IDENTITY

`BIGINT IDENTITY` is the only form: no seed, no increment, positive values that are never reused. Each ingestion task gets its own range across the nodes, so values are not in load order and gaps are normal. For a fixed value such as the -1 unknown member, `SET IDENTITY_INSERT dbo.t ON` (one table per session, column list required), insert, turn it off and run `DBCC CHECKIDENT('dbo.t', RESEED)`, which accepts no custom value. An identity column cannot be added with `ALTER TABLE`; rebuild the table with `CTAS`.

DELTA IDENTITY COLUMNS

`GENERATED ALWAYS AS IDENTITY` or `BY DEFAULT`, which also accepts explicit values, with optional `START WITH` and a non-zero `INCREMENT BY`. Values are unique and move in the step's direction, but are not contiguous. `BIGINT` only, not a partition column, not added or changed with `ALTER TABLE`, never updated. Declaring one disables concurrent transactions on the table, and Fabric engines cannot write to a table that has one.

HASH KEYS

`xxhash64(employee_id)`, or `sha2` for fewer collisions, gives the same key for the same input on every load, with nothing to coordinate between writers, and a fact can compute its dimension key before the dimension row exists. The price: a wider key, a small collision risk at 64 bits, and both engines must hash exactly the same bytes, so fix the type, case and trimming first. In a Type 2 dimension, hash the business key together with `valid_from`.

RULES FOR ANY SURROGATE KEY

Give it no meaning: do not sort by it to find load order or filter on its ranges. Keep a row for the unknown member so facts with a missing dimension still join. Store the business key beside it, and store the surrogate key in facts at load time.

RELATED CONCEPTS

SCD Type 2, Constraints the optimiser trusts, Order and determinism, One table, two engines.

SCD Type 1.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

A Type 1 dimension keeps only the current value. When an employee's job title or salary changes, the row is overwritten and the old value is gone. One `MERGE` per load does it: update the keys that exist, insert the ones that do not. Fabric Warehouse and Spark SQL run the same statement with small differences in syntax.

THE MERGE

Match on the business key, `ON t.employee_id = s.employee_id`, update `WHEN MATCHED` and insert `WHEN NOT MATCHED`. Four staged changes against a 19-row dimension give three updates (a new title and salary for 1002, a first salary for 1008, a correction of the -5000 for 1011) and one insert (1019): 20 rows. In the Warehouse, `MERGE` is generally available; it must end with a semicolon, has no `OUTPUT` clause, and its source cannot be a `VALUES` list. On Delta, `UPDATE SET *` and `INSERT *` save listing the columns.

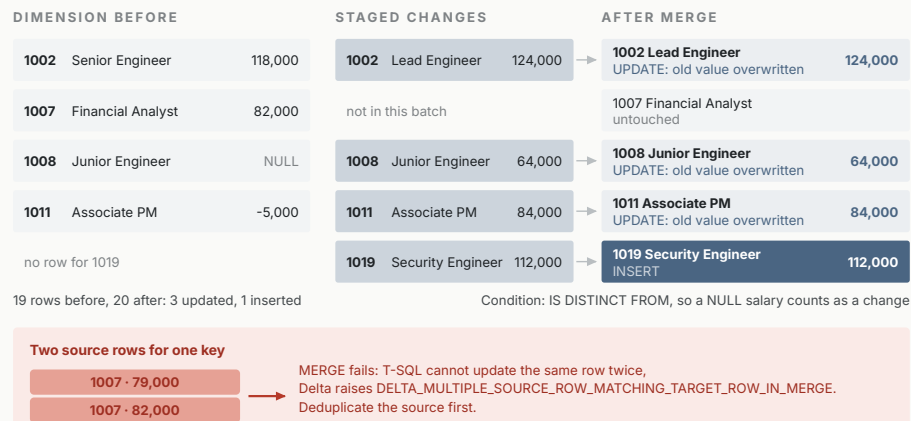


FIG. 7.3 A Type 1 load. The staged changes are merged on employee_id: three matched keys with different values are overwritten, one new key is inserted, and keys absent from the batch are untouched. A source with two rows for one key makes the MERGE fail.

UPDATE ONLY WHAT CHANGED

Add a condition to the matched branch: `WHEN MATCHED AND (t.salary IS DISTINCT FROM s.salary OR t.job_title IS DISTINCT FROM s.job_title)`. Unchanged rows are then left alone, so the affected row count means something and Delta rewrites fewer files. Use `IS DISTINCT FROM`, not `<>`: the first salary for 1008 replaces a `NULL`, and `<>` would see no change.

ONE SOURCE ROW PER KEY

If the source holds two rows for one key, as employee 1007 does, the `MERGE` fails. T-SQL refuses to update the same row more than once, and Delta raises `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`. Deduplicate the source first, keeping the latest change per key with `ROW_NUMBER()`, and order it by a change timestamp plus a tiebreaker so the same row wins every time.

DELETES

When the source is a full snapshot, `WHEN NOT MATCHED BY SOURCE THEN DELETE` removes keys that have disappeared; both engines support it, from Databricks Runtime 12.2 on Databricks. Do not use it with an incremental batch, because every key missing from the batch would be deleted. Many teams set an `is_deleted` flag instead, which keeps the key for facts that still point at it.

IN A PIPELINE

On Databricks, `AUTO CDC ... STORED AS SCD TYPE 1` applies changes into a streaming table in `SEQUENCE BY` order and drops an update that arrives after a later one. In Fabric you schedule the `MERGE` yourself, from a pipeline or a notebook.

RELATED CONCEPTS

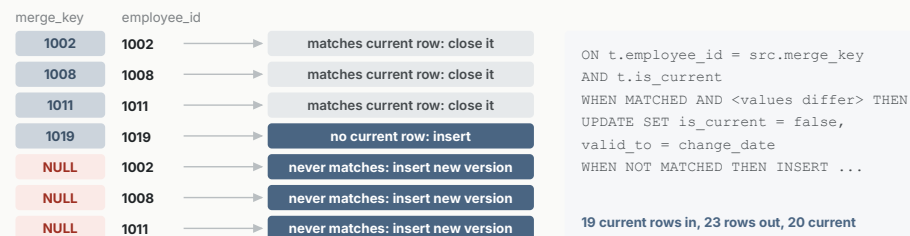
SCD Type 2, Order and determinism, NULL and three-valued logic, Join cardinality and fan-out.

SCD Type 2.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

A Type 2 dimension keeps every version. A change closes the current row, setting `valid_to` and `is_current = false`, and inserts a new row with its own surrogate key. Facts loaded before the change keep pointing at the old version, so a report shows what was true at the time.

MERGE SOURCE · 7 ROWS FROM 4 CHANGES



VERSIONS ON A TIME LINE

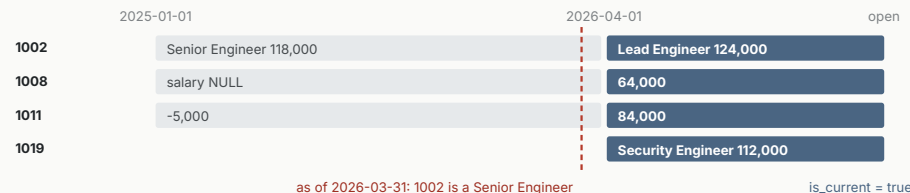


FIG. 7.4 A Type 2 load in one MERGE. Each changed row is fed in twice: with merge_key set to the business key it matches and closes the current version, with merge_key NULL it never matches and inserts the new one. Below, the versions on a time line, and what an as-of query returns.

THE TABLE

Each row carries a surrogate key, the business key, the tracked attributes, `valid_from`, `valid_to` (NULL or 9999-12-31 while current) and `is_current`. In the Warehouse the surrogate key is a `BIGINT IDENTITY` column: no seed or increment, values unique but not in order and with gaps between loads. On Delta it is `GENERATED ALWAYS AS IDENTITY`, also not contiguous, and Databricks warns that declaring one disables concurrent transactions on the table.

ONE MERGE, TWO ACTIONS

A changed key needs two things, closing the old row and inserting the new one, but `MERGE` acts once per target row. Feed each changed row in twice: once with a `merge_key` equal to the business key, which matches and closes the current row, and once with `merge_key` set to NULL, which never matches and so inserts the new version. New keys arrive once and insert. Four changes against 19 current rows make a 7-row source and leave 23 rows, 20 of them current. Running the same batch again changes nothing, because the NULL-key copies are built only for rows that differ from the current version.

TWO STATEMENTS INSTEAD

The other way is an `UPDATE` that closes the changed current rows, then an `INSERT` of the new versions. Wrap both in one transaction, `BEGIN TRAN ... COMMIT` in the Warehouse, so no reader sees a key with no current row between the two statements.

READING HISTORY

The current view is `WHERE is_current`. As of a date, keep the version with `d >= valid_from AND (d < valid_to OR valid_to IS NULL)`: employee 1002 is a Senior Engineer on 2026-03-31 and a Lead Engineer from 2026-04-01. Facts store the surrogate key current at load time, or join on the business key and the date range. Check the table after every load: `GROUP BY employee_id HAVING SUM(CASE WHEN is_current THEN 1 ELSE 0 END) <> 1` must return nothing.

LATE CHANGES

A change dated before the current version's `valid_from` does not fit the pattern: it belongs in the middle of the history, and the neighbouring rows' dates have to move. Sort the source by change date and apply one change per key per run, or on Databricks let `AUTO CDC ... STORED AS SCD TYPE 2` order the records by `SEQUENCE BY`; it adds `__START_AT` and `__END_AT` to a streaming table.

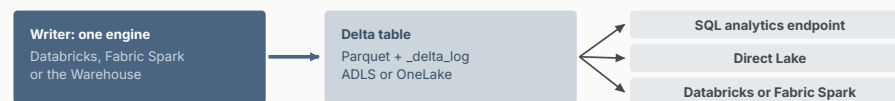
RELATED CONCEPTS

SCD Type 1, Window functions, Order and determinism, Join cardinality and fan-out.

One table, two engines.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN OPERATE

One Delta table can be written by one engine and read by others: Databricks writes and Fabric reads through a shortcut or a mirrored catalog, or the Warehouse writes and Spark reads its published log. Each engine supports a different set of Delta features, and a feature the writer turns on can hide a column, or the whole table, from a reader.



WRITTEN ON ONE SIDE, TROUBLE ON THE OTHER

TIMESTAMP_NTZ column	not supported by the SQL analytics endpoint or Direct Lake
Column mapping by ID	endpoint supports mapping by name only
V2 checkpoints	endpoint and lakehouse explorer do not list the table
Collations, VARIANT (Delta 4.x)	readable only by Fabric Spark notebooks and jobs
Identity column	Fabric engines cannot write to it
Row filter or column mask (UC)	table cannot be mirrored into Fabric
STRING, TIMESTAMP	varchar(8000) cut at 8 KB; datetime2; binary collation

Two engines writing one table must agree on every table feature: give each table one owner.

FIG. 8.1 One Delta table, one writer, several readers. Each reader supports a different set of Delta features, and a feature the writer turns on can hide a column or the whole table from a reader. Keep one engine writing each table and check what the readers support before you enable a feature.

ONE OWNER PER TABLE

Decide which engine writes each table; the others read. Fabric enforces part of this already: Warehouse tables are read-only from outside the Warehouse, and the SQL analytics endpoint takes no writes. Unity Catalog tables reach Fabric through shortcuts or a mirrored catalog, which copies metadata only and reads the data in place.

FEATURES THAT BREAK READERS

The SQL analytics endpoint and Direct Lake do not support `TIMESTAMP_NTZ`. The endpoint supports column mapping by name, not by ID, and does not list tables with V2 checkpoints. Delta 4.x collations and `VARIANT` are readable only by Fabric Spark. Fabric engines cannot write to a table with an identity column. Unity Catalog tables with row filters or column masks, streaming tables, materialised views and views cannot be mirrored.

TYPES CHANGE ON THE WAY

A lakehouse endpoint turns `STRING` into `varchar(8000)` and truncates longer values at 8 KB, and `TIMESTAMP` into `datetime2`, with a binary, case-sensitive collation on every string. The same SQL text can also mean different things: integer division, trailing spaces and where `NULL` sorts all differ between the engines, as Part 1 shows.

FRESHNESS IS UNEVEN

The endpoint trails a Spark commit by up to about a minute, the Warehouse publishes its Delta log after commit, a mirror follows its source every 15 seconds at best, and Direct Lake reads as of its last framing. A report joining two tables written by different engines can see them at different moments.

RELATED CONCEPTS

What the SQL analytics endpoint reads, How the Warehouse writes Delta, SQL permissions vs OneLake security, Type precedence and implicit conversion.

Parameters and SQL injection.

LAYER STORAGE TABLE MODEL LOAD **QUERY** COMPUTE GOVERN OPERATE

SQL injection happens when input is pasted into the text of a statement: a quote in the input ends the literal, and the rest runs as SQL. Parameters prevent it by sending values separately from the statement, so they can only ever be values. Object names cannot be parameters in plain SQL and need their own treatment.

HOW IT HAPPENS

Build `"... WHERE email = '" + x + "'"` and pass `x = "' OR 1=1 --"`: the statement becomes `WHERE email = "' OR 1=1 --"` and returns every row. The same risk sits in dynamic SQL inside procedures, in pipeline expressions that assemble SQL, and in notebooks that format strings into `spark.sql`.

CONCATENATED

```
sql = "... WHERE email = '" + x + "'"
x = "' OR 1=1 --"
... WHERE email = "' OR 1=1 --"
every row comes back
```

PARAMETERISED

```
... WHERE email = @email
@email = N'' OR 1=1 --'
compared as one string value
no row matches
```

EACH ENGINE**VALUES****OBJECT NAMES****T-SQL, Fabric**

```
EXEC sp_executesql @stmt,
N@email varchar(100)', @email
```

QUOTENAME(@t)
after an allow-list check; 128 chars max

Databricks SQL

```
WHERE email = :email
EXECUTE IMMEDIATE s USING x
```

IDENTIFIER(:tab)
from Runtime 13.3

In T-SQL, parameters also make plan reuse more likely, and on both they keep the SQL text stable in query history. Do not mix named and unnamed markers in one Databricks statement.

FIG. 8.2 Concatenated input becomes part of the statement, so a quote in it can rewrite the query. A parameter is sent separately and can only ever be a value. Object names cannot be parameters in plain SQL: check them against an allow list and quote them with `QUOTENAME` or `IDENTIFIER()`.

IN T-SQL

Use `sp_executesql` with a parameter list: `EXEC sp_executesql @stmt, N@email varchar(100)', @email = @input`. Microsoft's own warning on the page is to parameterise when you use it, and parameter markers also make plan reuse more likely. `EXEC(@sql)` over a concatenated string is the form to remove.

IN DATABRICKS SQL

Write `:email` or `?` markers, never both in one statement, and pass the values through the SQL Statement Execution API or `spark.sql(query, args=...)`. Databricks states that markers protect against injection by separating values from structure. From Runtime 18.0 a marker can stand wherever a literal of its type can. Inside a script, `EXECUTE IMMEDIATE sql USING ...` binds values the same way.

OBJECT NAMES

A table or column name cannot be a value. In T-SQL, check the name against `sys.tables` or an allow list, then wrap it in `QUOTENAME(@t)`, which returns `NULL` for input over 128 characters. In Databricks, `IDENTIFIER(:tab)`, from Runtime 13.3, turns a parameter into a name safely in queries, DDL, `MERGE` and `COPY INTO`.

WHERE IT MATTERS MOST

Dynamic SQL in a Databricks procedure declared `SQL SECURITY DEFINER` runs with the owner's authority, so injection there reaches whatever the owner can. Keep such procedures free of concatenation.

RELATED CONCEPTS

SQL scripting and procedures, SQL in CI/CD, SQL permissions vs OneLake security, Testing SQL and data quality.

Testing SQL and data quality.

LAYER STORAGE TABLE MODEL **LOAD** QUERY COMPUTE GOVERN **OPERATE**

A data test is a query that must return no rows: every row it returns is a problem. Run the set after each load and before anyone reads the table, and stop publishing when one fails. Databricks can enforce `NOT NULL` and `CHECK` at write time; everything else, and everything in Fabric, is a query you run.



A chip marks a check Databricks can also enforce at write. T-SQL = ignores trailing spaces, hence LIKE.

Write them once as dbt tests, SQL task steps or a stored procedure; the same SELECTs run on both engines.

FIG. 8.3 Tests as queries that must return no rows, run after every load and before anyone reads the table. Against the dataset, five checks catch the five problems planted in it. Databricks can enforce two of them at write time; in Fabric all of them are queries.

THE CHECKS THAT MATTER

Against the dataset: required values (`salary IS NULL` finds 1008 and 1017), unique keys (1007 twice), references (1018 in department 111), ranges (1011 at -5,000) and clean text (1015 with a trailing space). Add one current row per key for Type 2 tables, and row counts and totals reconciled with the source.

WRITE THEM FOR BOTH ENGINES

Use `NOT EXISTS` for references; `NOT IN` fails on a `NULL`, and `LEFT ANTI JOIN` is Spark only. T-SQL `=` ignores trailing spaces, so test for them with `LIKE '% '`. Avoid `TOP`, integer division and collation-dependent comparisons in tests shared by both engines.

WHERE THEY RUN

As steps after the load: a stored procedure, a SQL task in a Databricks job, or dbt tests such as `unique`, `not_null` and `relationships`, which generate the same queries through the dbt-fabric or dbt-databricks adapter. Microsoft's advice for returned rows is to treat them as candidates for review, confirm the rule with the data owner, and fix the source rather than update facts in place.

TEST THE SQL ITSELF

For a `MERGE` or a Type 2 load, keep a small fixed input, like the 20-row dataset, and the rows you expect. Run the statement, then compare with `EXCEPT` in both directions plus a count: two empty results and equal counts mean the output matches.

RELATED CONCEPTS

Constraints the optimiser trusts, SCD Type 2, NULL and three-valued logic, SQL in CI/CD.

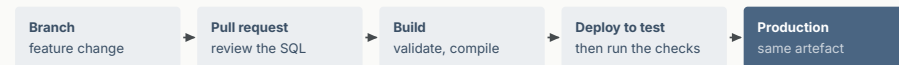
SQL in CI/CD.

LAYER STORAGE TABLE MODEL LOAD QUERY COMPUTE GOVERN **OPERATE**

Tables, views and procedures are code: kept in Git, reviewed, built, deployed to a test environment, checked, then promoted. Fabric builds a warehouse from a SQL database project; Databricks deploys SQL files and the jobs that run them with bundles. The hard parts are what is not plain DDL: permissions, data changes and dependencies between items.

FABRIC WAREHOUSE

Git integration connects a workspace to Azure DevOps or GitHub and commits the warehouse as a SQL database project. In VS Code, the SQL Database Projects extension uses DacFx to build a `.dacpac`, compare schemas and publish. Deployment pipelines promote dev to test to prod, or SqlPackage does it from your own pipeline.

**FABRIC WAREHOUSE**

Git integration: workspace ↔ Azure DevOps or GitHub
 SQL database project: .sqlproj, DacFx, a .dacpac
 deployment pipelines: dev → test → prod
 or SqlPackage in your own pipeline

permissions need a post-deployment script
 SQL analytics endpoints are not versioned
 three-part names to the same warehouse can
 fail a commit

DATABRICKS

Git folders for .sql files and notebooks
 Declarative Automation Bundles: databricks.yml
 bundle validate, deploy, run per target
 jobs with SQL tasks on a serverless or pro warehouse

a SQL file task can hold several statements
 procedures: CREATE OR REPLACE in the files
 formerly Databricks Asset Bundles

Tests are SQL too: the zero-row assertions run in the test stage on both platforms.

FIG. 8.4 The same promotion path on both platforms: SQL in Git, a build that validates it, deployment to test, checks, then production. Fabric builds a warehouse from a SQL database project; Databricks deploys SQL files and the jobs that run them from a bundle.

FABRIC CAVEATS

Permissions need a post-deployment script. SQL analytics endpoints are not under version control, and dependencies between warehouses and endpoints are not supported in these workflows. Commits are per warehouse, not per object. A three-part name to an object in the same warehouse, a changed `IDENTITY` column or an old `Microsoft.Build.Sql` version pinned in the `.sqlproj` can each make a commit fail.

DATABRICKS

Keep `.sql` files in Git folders. Declarative Automation Bundles, formerly Databricks Asset Bundles, describe jobs, pipelines and dashboards in `databricks.yml`, and `validate`, `deploy` and `run` push them to each target workspace. A job's SQL task runs a file of several statements on a serverless or pro warehouse, with parameters. Bundles deploy files and jobs, so the SQL has to be safe to run again: `CREATE OR REPLACE` for views and procedures, `IF NOT EXISTS` for tables.

WHAT THE TEST STAGE RUNS

The zero-row assertions from Testing SQL and data quality, fixed-input tests for each `MERGE`, and a check on the key queries' cost, found by label in query insights or by statement in `system.query.history`.

RELATED CONCEPTS

Testing SQL and data quality, SQL scripting and procedures, Parameters and SQL injection, Time travel, clones and snapshots.

What a query costs.

LAYER STORAGE TABLE MODEL LOAD QUERY **COMPUTE** GOVERN OPERATE

On Fabric, every Warehouse and SQL analytics endpoint query consumes capacity units in proportion to the compute allocated to it, so a query has a cost you can estimate. On Databricks, usage is recorded per SQL warehouse, not per query, so a query's cost is a share you allocate. The levers are the same on both: read less, reuse results, and do not keep compute running for nothing.

FABRIC WAREHOUSE AND SQL ANALYTICS ENDPOINT

consumption = active vNodes x active time
 1 vNode = 4 VCores
 1 Warehouse VCore = 0.538 CU
 allocated in 1-minute windows, reported per second

one query:
`allocated_cpu_time_ms` 120,000 = 120 VCore-s
 120 x 0.538 = about 65 CU(s)
 about one second of an F64 (64 CU)

DATABRICKS SQL WAREHOUSE

usage in DBUs, per warehouse:
`system.billing.usage`, `usage_metadata.warehouse_id`
 prices by SKU: `system.billing.list_prices`

no bill per query; to share it out, split the
 warehouse's usage by `total_task_duration_ms`

auto stop limits time running with nothing to do

WHAT COSTS MORE THAN IT LOOKS

Fabric	queries on catalog views and DMVs are billed; a cross-database query is billed to the item it ran from
Fabric	background work such as compaction also uses Warehouse compute
Both	a cold read from storage: judge cost on a warm run
Both	an AI function per row in a report query

Cheaper by design: prune files, select only needed columns, let caches hit, right-size, stop idle compute.

FIG. 8.5 Fabric charges the capacity for Warehouse compute by the VCore-second, so each query has a cost you can estimate from query insights. Databricks records usage per SQL warehouse, not per query; a query's share is an allocation. On both, the cheapest query is the one that reads less or does not run.

FABRIC: VCORE-SECONDS

Consumption is active vNodes times active time, where a vNode is four VCores, allocated in one-minute windows and reported per second. One Warehouse VCore is 0.538 capacity units. Query insights' `allocated_cpu_time_ms` divided by 1,000 is VCore-seconds, as Microsoft's own example query computes it: a query at 120,000 ms uses about 120 × 0.538, or 65 CU(s), roughly one second of an F64. Most Warehouse work counts as background and is smoothed over 24 hours.

FABRIC: WHERE IT SHOWS

The Capacity Metrics app reports Warehouse Query, SQL Endpoint Query and Warehouse Snapshot Query operations; query insights shows what ran in the busy window. A cross-database query is charged to the item it ran from, queries on catalog views and DMVs are billed, and system work such as compaction appears too.

DATABRICKS: WAREHOUSE USAGE

`system.billing.usage` records usage per `usage_metadata.warehouse_id` and SKU, priced by `system.billing.list_prices`. There is no line per query; to share a warehouse's usage out, split it by each query's `total_task_duration_ms` from `system.query.history`. Size and cluster count set how fast usage accrues, and auto stop limits the time a warehouse runs with nothing to do.

CHEAPER BY DESIGN

Read less: prune files, name only the columns you need, filter before joining, avoid fan-out. Do not run at all: let result caches hit, and store AI function outputs and heavy aggregations instead of recomputing them in every report query.

RELATED CONCEPTS

Workload management and isolation, SQL warehouse sizing and scaling, Result set caching, AI functions in SQL.

IX

Reference.

Two pages that start from where you are. Diagnose by symptom goes from an error message or a surprising result to its cause and fix. Which engine when goes from a task to the engine or item that suits it. Both point back to the pages that explain why.

IX	Reference	
49	Diagnose by symptom	54
50	Which engine when	55

Diagnose by symptom.

Find the symptom you are looking at. Each entry gives the usual cause, where to confirm it, the fix, and the pages that explain the mechanism.

Invalid column name for an alias in WHERE

Cause: the alias is created in `SELECT`, after `WHERE` runs; T-SQL allows it only in `ORDER BY`.

Look: the clause that names the alias. **Fix:** repeat the expression, or compute it in a subquery or CTE and filter outside. See: Logical processing order.

Row count grew after a join

Cause: the join key repeats on both sides, so rows multiply. **Look:** `GROUP BY` key `HAVING COUNT(*) > 1` on each input. **Fix:** deduplicate or aggregate the many side before joining. See: Join cardinality and fan-out; Bags, sets and DISTINCT.

NOT IN returns no rows at all

Cause: the subquery returns a `NULL`, so every comparison is unknown. **Look:** the subquery column for `NULL`s. **Fix:** use `NOT EXISTS`. See: Semi joins, anti joins and NOT IN; NULL and three-valued logic.

A division gives 0 in the Warehouse and 0.5 in Spark

Cause: integer divided by integer truncates in T-SQL; Spark returns a double. **Look:** the operand types. **Fix:** cast one side to `decimal` first. See: Type precedence and implicit conversion.

Equal-looking strings do not match, or match wrongly

Cause: collation and trailing spaces: T-SQL `=` ignores trailing spaces and collations differ in case sensitivity. **Look:** the column collations and `LEN` against `DATALENGTH`. **Fix:** trim at load time and state the collation. See: Collation and case sensitivity.

Results change from run to run

Cause: a sort, `ROW_NUMBER`, `TOP` or `LIMIT` without a unique ordering. **Look:** ties in the sort keys. **Fix:** add a unique column as the last sort key. See: Order and determinism.

A running total repeats on tied rows

Cause: the default frame is `RANGE`, which treats ties as one peer group. **Look:** the window definition. **Fix:** write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. See: Window frames: ROWS vs RANGE.

First run slow, second run fast

Cause: a cold start: data read from OneLake, statistics created, nodes added. **Look:** `data_scanned_remote_storage_mb` above zero. **Fix:** judge performance on warm runs; update statistics after big loads. See: Statistics and cardinality estimates; Result set caching.

One or more non-scalable operation is detected

Cause: a step that must run on one node, such as `TOP` or a global sort, receives too many rows. **Look:** `is_distributed` and the plan. **Fix:** filter and aggregate earlier; `OPTION (FORCE DISTRIBUTED PLAN)` when the semantics allow. See: Distributed plans and data movement.

Snapshot isolation transaction aborted (error 24556)

Cause: two transactions changed the same table; the second to commit loses. **Look:** overlapping writers in the orchestration. **Fix:** one writer per table at a time, with retry and backoff. See: Snapshot isolation and write conflicts.

Queries wait on a lock in the Warehouse

Cause: an open transaction or DDL holds a table lock. **Look:** `sys.dm_tran_locks` with `request_status = 'WAIT'` and `blocking_session_id`. **Fix:** commit or roll back the blocker; an Admin can `KILL` it. See: Snapshot isolation and write conflicts; Multi-table transactions.

This query was rejected due to current capacity constraints

Cause: the query cannot run within the burst limit of the SKU. **Look:** query insights and `sql_pool_insights`. **Fix:** read less data first, then consider a larger SKU. See: Workload management and isolation.

Spark wrote rows that T-SQL does not show yet

Cause: the SQL analytics endpoint syncs from the Delta log in the background. **Look:** the lag after the write. **Fix:** wait, or refresh the metadata. See: What the SQL analytics endpoint reads.

A column or table is missing from the endpoint

Cause: an unsupported type such as `TIMESTAMP_NTZ`, column mapping by ID, or a table outside `/Tables`. **Look:** the Delta table's properties and location. **Fix:** change the type or feature on the writer, or add a shortcut. See: One table, two engines; What the SQL analytics endpoint reads.

A Direct Lake report is slower than expected

Cause: DirectQuery fallback: SQL security, a view or a guardrail breached. **Look:** `EVALUATE TABLETRAITS()`. **Fix:** fix the cause, and develop with `DirectLakeOnly`. See: Direct Lake fallback.

Databricks queries wait before they start

Cause: the warehouse is at capacity or still starting. **Look:** `waiting_at_capacity_duration_ms` and `waiting_for_compute_duration_ms`. **Fix:** raise the maximum clusters, or use serverless. See: SQL warehouse sizing and scaling.

MERGE fails: several source rows match one target row

Cause: the source holds duplicates of a key. **Look:** `GROUP BY key HAVING COUNT(*) > 1` on the source. **Fix:** keep one row per key with `ROW_NUMBER()` before the `MERGE`. See: SCD Type 1; SCD Type 2.

Wrong results after adding RELY to a key

Cause: the data breaks the key, and the optimiser removed a `DISTINCT` or a join. **Look:** a duplicate check on the key. **Fix:** remove duplicates, or drop `RELY`. See: Constraints the optimiser trusts.

A Spark user sees rows that SQL security hides

Cause: SQL rules apply only through the SQL engine; ReadAll reads the files. **Look:** the item permissions and the endpoint's access mode. **Fix:** use OneLake security roles and user identity mode. See: SQL permissions vs OneLake security.

Recursive CTE fails in the Warehouse

Cause: Fabric Warehouse does not support recursive queries. **Look:** the error on `WITH ... UNION ALL` referencing itself. **Fix:** loop over levels into a table, or run the query in Spark SQL. See: Recursive queries.

Which engine when.

Find the task you are choosing an engine or item for. Each entry names the choice, the reason, and the pages that back it.

A star schema served to Power BI

Use: a Fabric Warehouse, read by Direct Lake. **Why:** T-SQL DDL and DML, transactions across tables and automatic maintenance, with no import refresh. See: Warehouse, endpoint and Lakehouse; Direct Lake fallback.

Transform in Spark, read in T-SQL

Use: a lakehouse and its SQL analytics endpoint. **Why:** Spark owns the writes, T-SQL reads the same Delta tables with no copy. See: What the SQL analytics endpoint reads.

Upserts and slowly changing dimensions in SQL

Use: `MERGE` on either engine. **Why:** the same statement shape on both; the Warehouse fails concurrent writers per table. See: SCD Type 1; SCD Type 2; Snapshot isolation and write conflicts.

Several tables that must change together

Use: a Warehouse transaction, or a Databricks transaction on managed tables. **Why:** the Warehouse allows DDL inside; Databricks does not but detects conflicts per row in `BEGIN ATOMIC`. See: Multi-table transactions; Multi-statement transactions.

Hierarchies and graph walks

Use: Spark SQL `WITH RECURSIVE`. **Why:** the Warehouse has no recursive queries. See: Recursive queries.

An application database with analytics on the side

Use: a Fabric SQL database. **Why:** OLTP on the Azure SQL engine, with a mirror for analytics that does not load the database. See: SQL database and mirroring.

Bringing an external database into Fabric

Use: mirroring. **Why:** no pipelines to build; replication compute is free. See: SQL database and mirroring.

Databricks tables queried from Fabric

Use: a shortcut or a mirrored Unity Catalog, read through an endpoint. **Why:** the data stays in place; check the Delta features first. See: One table, two engines.

Many concurrent dashboard queries

Use: a serverless SQL warehouse with enough clusters, or the Warehouse with result set caching. **Why:** concurrency comes from clusters or pools, and repeated queries from caches. See: SQL warehouse sizing and scaling; Result caches; Workload management and isolation.

Business metrics shared by many tools

Use: Databricks metric views, or a Power BI semantic model. **Why:** one definition evaluated at any grouping. See: Metric views.

Classifying or extracting from text

Use: AI functions, stored once as columns. **Why:** Databricks has the wider set and `ai_query`; Fabric's are in preview. See: AI functions in SQL.

Recovery and a stable reporting copy

Use: Warehouse time travel, clones, snapshots and restore points. **Why:** all read the retained history without copying files. See: Time travel, clones and snapshots.

Procedural SQL with loops and error handling

Use: T-SQL procedures, or Databricks SQL scripting and procedures. **Why:** both work; what commits when differs. See: SQL scripting and procedures.

Security that every reader respects

Use: OneLake security roles with the endpoint in user identity mode. **Why:** SQL rules apply only through the SQL engine. See: SQL permissions vs OneLake security.

CONCLUSION

How seniors, leads and architects think.

If you can say what a query means, how many rows each step returns and which engine runs it, most production problems point to their own cause.

- 01 Read a query in logical order and most surprises explain themselves.
- 02 Know which side of a join is unique before you trust a count.
- 03 Check types, collation and NULL wherever data crosses from one engine to the other.
- 04 Give every table one writer, and test it after every load.

Version 4.0 joins the PySpark books in one series: this handbook for concepts, the **SQL Cheat Sheet 4.0** for query patterns, and **The PySpark Practitioner's Handbook 4.0** for the engine internals both share, all scoped to Fabric and Databricks.

FREE

Sanjay Chandra

The Databricks + Fabric guy on LinkedIn

WEBSITE

www.ssanjaychandra.com

GITHUB

github.com/ssanjaychandra123

LINKEDIN

linkedin.com/in/ssanjaychandra

EMAIL

ssanjaychandra@outlook.com

This reference owes its shape to the SQL, Delta Lake, Databricks and Microsoft Fabric communities whose documentation, blogs, conference talks and source code make the platforms legible. Specific gratitude to the engineers and writers maintaining Apache Spark, Delta Lake and the SQL Server documentation, and to the product teams at Databricks and Microsoft whose technical writing was a continuous reference.

Claude (Anthropic) was used as a writing assistant while compiling, reviewing and red-teaming this reference. Errors are mine alone; corrections via the channels above are welcome.